CS 6650 - Homework 10

Table of Contents

Replication: Leader–Follower and Leaderless Key-Value Stores ***2***

1. System overview **2**

2. How the code works **2**

2.1 Core modules and what they do 2
2.2 Leader–follower: what happens on a write 3
2.3 Leader–follower: what happens on a read 3
2.4 Leaderless: what happens on a write and read 3
2.5 Error handling and tricky implementation details 3
2.6 Showing understanding of the AI-assisted code 4

3. Test strategy **4**

4. Experimental setup **4**

5. Results and discussion **5**

5.1 Which leader–follower profile wins at each write ratio? 5
5.2 Mechanism-based analysis 5
Setup (one sentence) 5
Which leader–follower profile "wins" on throughput (ops in 50s) per write ratio? 5
Reasoning (tie to mechanism, not graph shapes) 5
Throughput vs write fraction (leader–follower) 5
Staleness 5
Application fit (which database style for what workload?) 6
Figures 6

6. Figures **7**

lf-r3w3-w0.01 7
lf-r3w3-w0.1 8
lf-r3w3-w0.5 10
lf-r3w3-w0.9 11
lf-w1r5-w0.01 13
lf-w1r5-w0.1 14
lf-w1r5-w0.5 16
lf-w1r5-w0.9 17
lf-w5r1-w0.01 19
lf-w5r1-w0.1 20
lf-w5r1-w0.5 22
lf-w5r1-w0.9 23
ll-w0.01 25
ll-w0.1 26
ll-w0.5 28
ll-w0.9 29

Replication: Leader–Follower and Leaderless Key-Value Stores

AI / tooling disclosure: Implementation, infrastructure (Docker, Terraform, ECR/EC2), load-test automation, plotting, and report generation were produced with assistance from an AI coding agent. The code and measurements were then traced, tested, deployed, and checked module-by-module so the team can explain what every important component does.

1. System overview

- The system runs five replicas and uses the same ordered PEERS list on every process. NODE_INDEX selects which entry is the current node.
- The store is in-memory and versioned. Every local write increments a per-key logical version; replicated writes only overwrite the local copy if the incoming version is at least as new.
- Leader–follower mode treats peer 0 as the leader. Only the leader accepts client writes, while followers proxy quorum reads to the leader.
- Leaderless mode treats every node symmetrically. The node that receives a write becomes that write's coordinator and pushes the value to all other replicas.
- The assignment's artificial sleeps are deliberate. They make quorum choices visible in latency histograms and stale-read behavior.

2. How the code works

2.1 Core modules and what they do

Question	Answer
internal/store/store.go	Thread-safe in-memory map. WriteLocal assigns monotonically increasing versions; PutVersion applies replicated updates only if the incoming version is new enough.
internal/kvsrv/peer_client.go	Wraps internal HTTP RPCs for replicate, peer-read, and cluster-read. Non-200 responses are turned into Go errors so the coordinator can decide whether to fail fast or log asynchronously.
internal/kvsrv/lf.go	Implements leader–follower routing, quorum-specific read/write behavior, async replication for W=1 and part of W=3, and leader-side cluster reads.
internal/kvsrv/leaderless.go	Implements symmetric W=N write coordination and R=1 local reads, which intentionally allows temporary staleness during propagation.
cmd/loadtest/main.go	Generates read/write mixes, tracks latency, records stale reads by comparing returned versions against the client's last observed write version for the same key, and saves samples to JSON.
internal/kvsrv/cluster_test.go	Starts in-process five-node clusters and checks the main behavioral claims: rejection of follower writes, eventual propagation, local lag windows, and successful quorum reads.

2.2 Leader–follower: what happens on a write

- A client sends POST /kv/set. If the request lands on a follower, the server immediately returns 403 because only the leader is allowed to coordinate writes.
- The leader validates the JSON payload and rejects an empty key with 400.
- The leader calls WriteLocal, which stores the new value locally and increments the per-key version.
- The leader then applies the selected quorum profile.

- w5r1: the leader synchronously replicates to all four followers. If any follower replication fails, the write returns 500. This gives the strongest write visibility but the highest write latency.
- w1r5: the leader returns 201 as soon as the leader's local commit succeeds. Replication to followers continues in a background goroutine. Failures in that async tail are logged instead of being sent back to the already-completed client request.
- r3w3: the leader blocks until two followers acknowledge, which means three replicas in total (leader plus two followers) have the write. The remaining followers are updated asynchronously.

The tricky part here is that success means different things for different profiles. In w5r1, success means every replica has acknowledged. In w1r5, success only means the leader committed locally, so a fresh read from a follower's local state can still be stale until background replication catches up.

2.3 Leader–follower: what happens on a read

- A client can send GET /kv/get to any leader–follower node.
- If the request arrives at a follower, that follower forwards the read to the leader's internal cluster-read endpoint instead of answering from local state.
- The leader executes the R rule for the active profile and returns the newest version it sees.

- w5r1 (R=1): the leader reads only its own local store, so reads are cheap.
- w1r5 (R=5): the leader reads all five replicas in parallel, including itself, and picks the entry with the highest version number.
- r3w3 (R=3): the leader reads three replicas and still resolves conflicts by highest version.

The subtle point is that the merge rule is version-based, not 'first response wins'. That is why the versioned store matters: even if some replicas lag, the read path can prefer the newest visible copy.

2.4 Leaderless: what happens on a write and read

- Any node can receive POST /kv/set and becomes the coordinator for that request.
- That node writes locally first, which creates the next version for the key on the coordinator.
- It then replicates sequentially to every other replica. Because this mode is W=N, any replication failure returns 500 to the client.
- GET /kv/get does not do quorum reads in leaderless mode. It simply returns the local node's current value, which is why stale reads are possible during the replication window.

Leaderless is therefore simple and symmetric, but it trades consistency for availability and low local read cost. The load tests show exactly that: good local responsiveness, but stale reads can appear because replicas do not reconcile on the read path.

2.5 Error handling and tricky implementation details

- Bad JSON and empty keys produce 400s early so malformed requests do not enter replication logic.
- Writes sent to a follower in leader–follower mode return 403 immediately.
- PeerClient treats non-success HTTP responses as errors, which keeps quorum code simple: synchronous paths fail the request, asynchronous paths log the error.

- Artificial delays are inside the server logic, not the load tester, so they affect real coordination paths and not just the client view.
- Async replication goroutines are tracked with a WaitGroup so tests and clean shutdowns do not leave background work dangling.
- The store applies replicated writes by version order. That avoids overwriting newer data with an older replicated message if messages arrive late.

2.6 Showing understanding of the AI-assisted code

The important invariants are small enough to explain without reading the code line-by-line: peer 0 is the only write leader in leader–follower mode; local writes create monotonically increasing versions; replicated writes are version-checked; w5r1 is synchronous on every follower, w1r5 acknowledges early and replicates later, r3w3 waits for a middle quorum; leaderless uses W=N and R=1; and the load tester marks a read stale when its returned version is behind the client's last known write version for the same key. These are the design rules every teammate should be able to explain before submission.

3. Test strategy

- The tests build real in-process five-node clusters instead of mocking quorum behavior.
- TestLeaderFollower_W5R1_ConsistentAfterWrite checks that a leader write is visible through /kv/get after synchronous replication completes.
- TestLeaderFollower_LocalReadDuringReplication intentionally probes /kv/local_read under w1r5 to show that follower-local state can lag while async replication is still running.
- TestLeaderless_EventualConsistencyWindow checks that non-coordinator replicas can miss a key during replication and then converge after the write finishes.
- Additional tests cover rejection of follower writes, empty keys, and the r3w3 read/write path.

Recommended command: go test ./... -race -timeout 120s

4. Experimental setup

Cloud load tests (50s each, 8 workers) vs terraform Elastic IP. LF quorum switched on EC2 via SSM (scripts/aws_ec2_set_lf_quorum.sh); full sweep: scripts/aws_loadtest_cloud_all_lf.sh

EIP: terraform output -raw kv_public_ip (this run: 35.174.237.75)

Leader–follower w5r1: 1421 / 1221 / 536 / 319 ops @ w 0.01,0.1,0.5,0.9 — stale 0
Leader–follower w1r5: 843 / 1538 / 1825 / 2160 ops — stale 0
Leader–follower r3w3: 1273 / 1259 / 797 / 549 ops — stale 0

Leaderless: 1841 / 1223 / 524 / 304 ops — stale 6, 4, 0, 0 (writes 1%, 10%, 50%, 90%)

5. Results and discussion

5.1 Which leader–follower profile wins at each write ratio?

Write ratio	w5r1 ops	w1r5 ops	r3w3 ops	Winner
1%	1421	843	1273	w5r1
10%	1221	1538	1259	w1r5
50%	536	1825	797	w1r5
90%	319	2160	549	w1r5

This table answers the rubric's direct question: in this AWS run, w5r1 is best only when writes are extremely rare (1%), while w1r5 wins once writes become common enough that synchronous W=5 dominates the total cost of completed operations.

5.2 Mechanism-based analysis

Setup (one sentence)

- Five-node leader–follower and five-node leaderless stacks on one EC2 host behind an Elastic IP; load generators ran from a laptop against public HTTP ports; LF quorum was switched across w5r1 / w1r5 / r3w3 via SSM editing docker-compose on the instance.

Which leader–follower profile "wins" on throughput (ops in 50s) per write ratio?

- 1% writes: w5r1 (1421) > r3w3 (1273) > w1r5 (843). Reads dominate; R=1 (leader-local) avoids read fan-out, so w5r1 leads. w1r5 pays R=5 on every cluster-read, so it is slowest.
- 10% writes: w1r5 (1538) > r3w3 (1259) > w5r1 (1221). Even with mostly reads, w1r5 wins on *count of completed ops* here: cheap leader commits on the minority writes avoid the long W=5 write path that w5r1 pays whenever a write happens; the mix shifts total work enough that leader latency dominates. (Intuition: r3w3 stays in the middle; w5r1 pays full replica sync on each write.)
- 50% writes: w1r5 (1825) >> r3w3 (797) > w5r1 (536). Cheap write ack (W=1) dominates when half the ops are writes.
- 90% writes: w1r5 (2160) >> r3w3 (549) > w5r1 (319). w5r1 collapses because every write waits for all followers synchronously.

Reasoning (tie to mechanism, not graph shapes)

- w5r1 minimizes read path cost (R=1) but maximizes write path cost (W=5) — best when writes are rare.
- w1r5 minimizes leader write return time (W=1) but pays R=5 read fan-out — still wins at high write fractions in this experiment because synchronous W=5 on w5r1 is the dominant bottleneck once writes are common.
- r3w3 balances W=3 and R=3 — typically middle throughput; good when you want a compromise between the two extremes.

Throughput vs write fraction (leader–follower)

- w5r1 (W=5,R=1): throughput drops sharply as write ratio rises (1421 → 319 ops in 50s) because every write pays for synchronized replication to all followers before ack.
- w1r5 (W=1,R=5): reads dominate cost (parallel fan-out + simulated fetch delay); writes return quickly after the leader commit, so at high write ratios aggregate ops can exceed w5r1 (2160 ops at 90% writes vs 319 for w5r1).
- r3w3: sits between the extremes; writes are cheaper than w5r1 but reads still do a three-way merge, so curves reflect a balanced quorum.

Staleness

- Leader–follower: 0 stale reads in this AWS run for all three profiles and all write ratios—reads go through the leader's quorum/merge path as implemented.

- Leaderless: R=1 local reads allow temporary lag behind the writer's replica; observed stale reads at low write mix (6 at 1% writes, 4 at 10%) when reads are frequent relative to completed replication; 0 stale at 50% and 90% writes where the workload behaves more write-heavy or replication catches up within the client's hot-key window.

Application fit (which database style for what workload?)

- w5r1 (strong writes, cheap reads): Workloads with infrequent writes and a need for linearizable-or-strong reads without fan-out — e.g. rarely-updated reference data, configuration, or "read-mostly" catalogs where write latency is acceptable.
- w1r5 (cheap writes, expensive quorum reads): Write-heavy or ingest pipelines where low write latency matters and you can pay for read reconciliation (analytics, logging metadata, dashboards that can tolerate slower reads or run off-peak).
- r3w3: Balanced OLTP-style mixes where you want better availability / latency tradeoffs than all-replica writes but stronger reads than a single node — typical "middle quorum" for general services.
- Leaderless W=N, R=1: Low read latency, partition-tolerant scale-out memcached/dynamo-flavored use cases where eventual consistency and occasional stale reads are acceptable (feature flags, social counters, session caches). Not for balances or inventory without application-level versioning.

Figures

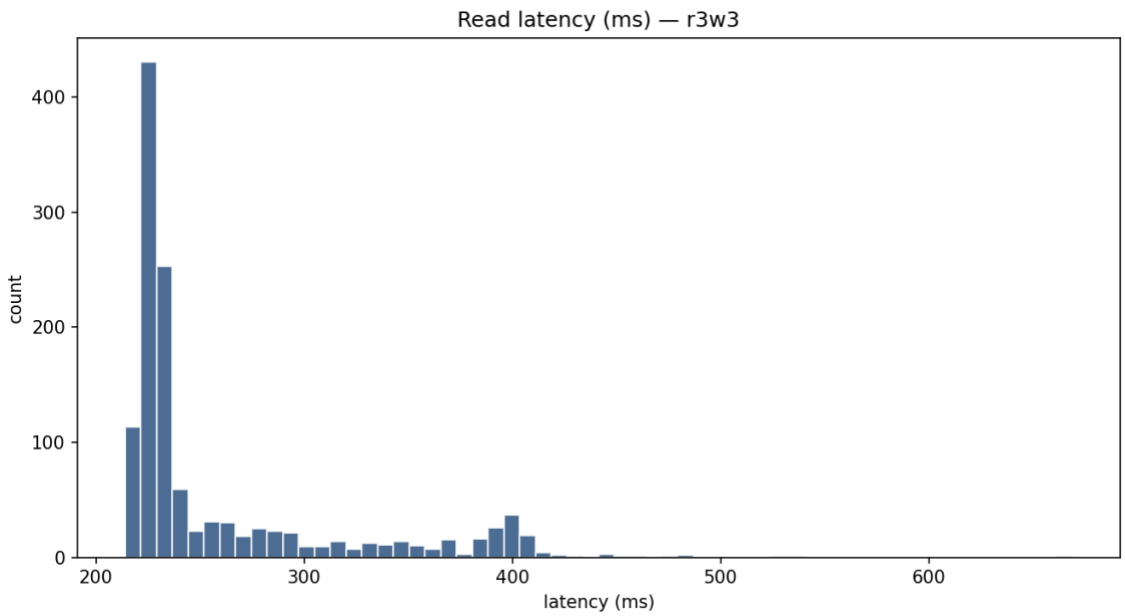
- Histograms: results/aws/figs/*-{read-latency,write-latency,rw-interval}.png (and local results/figs for laptop Docker if you compare environments).

6. Figures

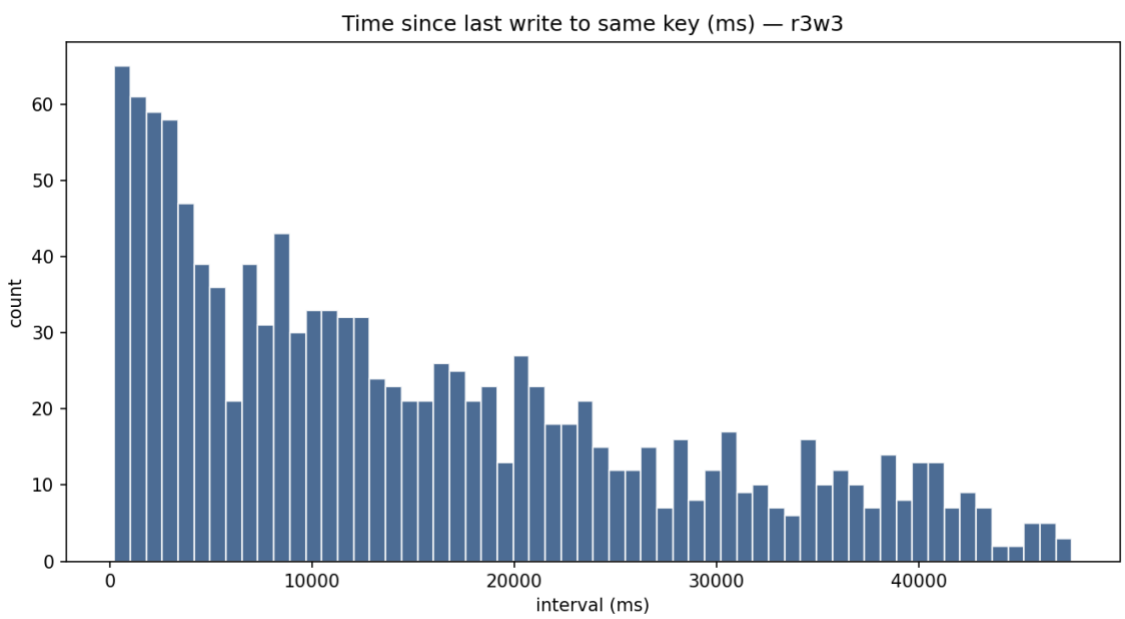
The following figures cover read latency, write latency, and time since the last write to the same key for all write ratios in the AWS sweep.

lf-r3w3-w0.01

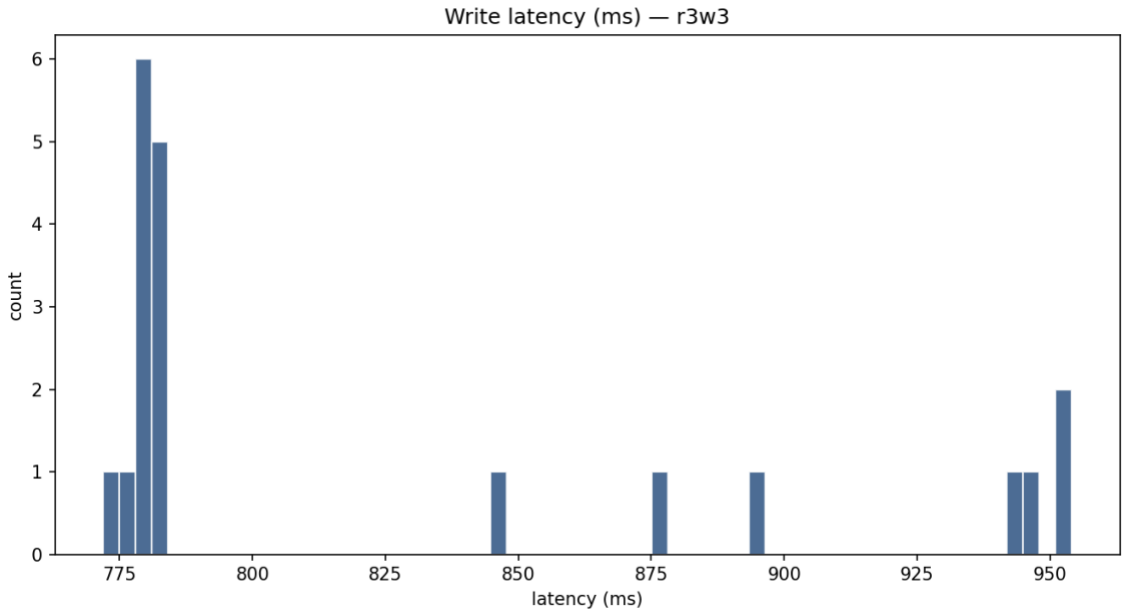
lf-r3w3-w0.01-read-latency.png



lf-r3w3-w0.01-rw-interval.png

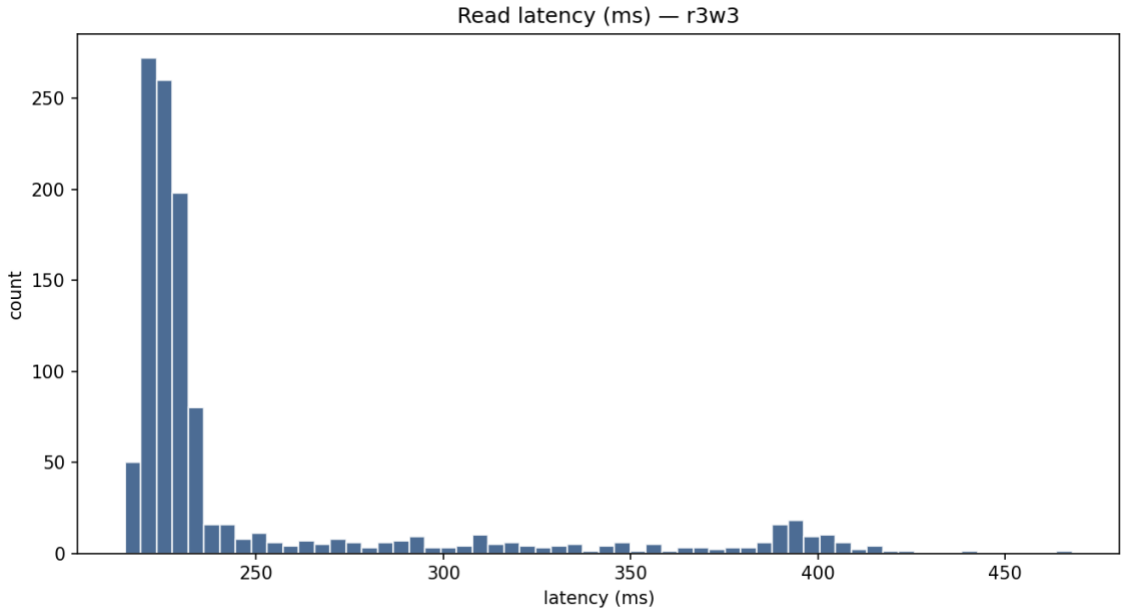


lf-r3w3-w0.01-write-latency.png

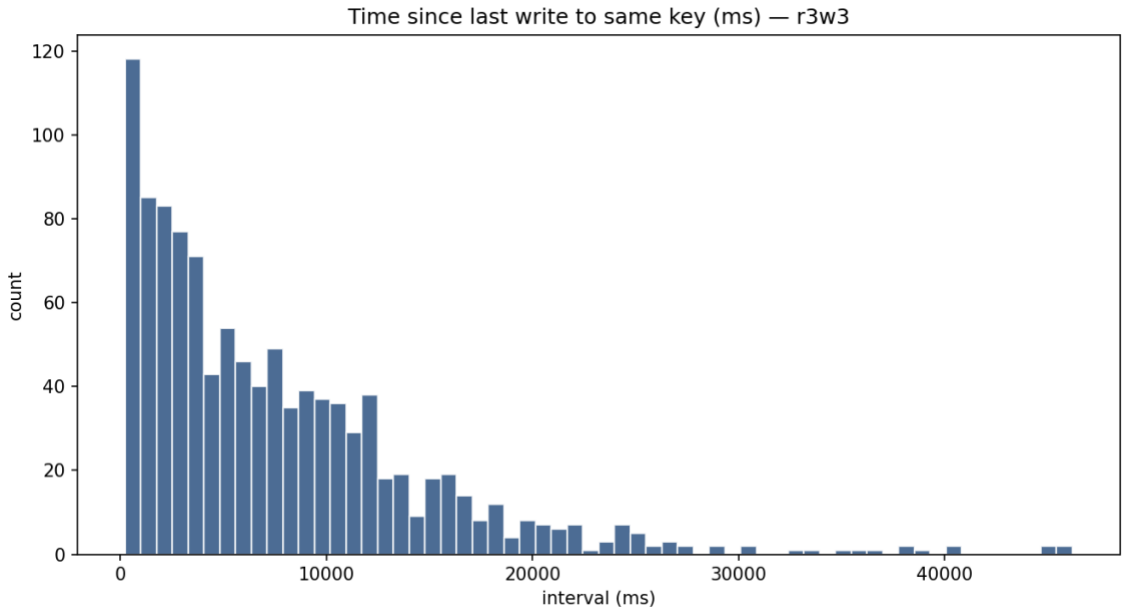


lf-r3w3-w0.1

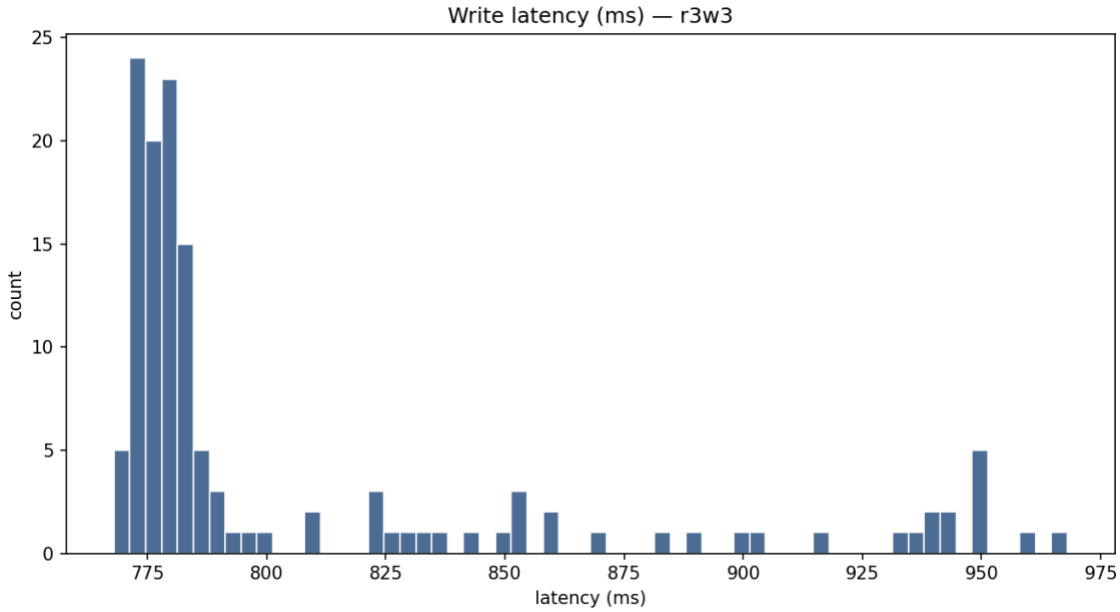
lf-r3w3-w0.1-read-latency.png



lf-r3w3-w0.1-rw-interval.png

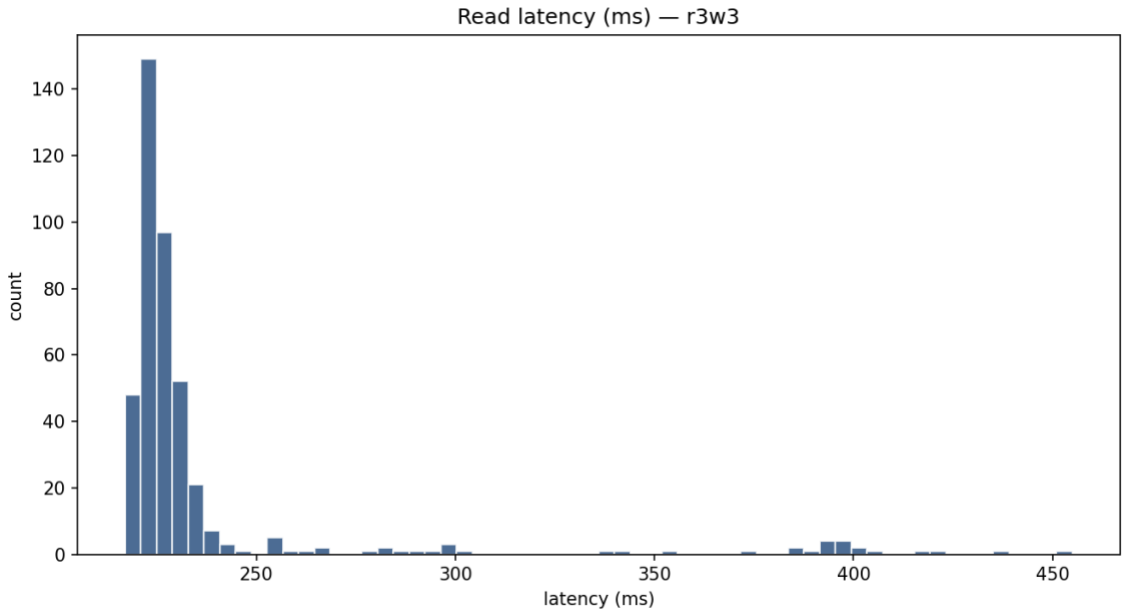


lf-r3w3-w0.1-write-latency.png

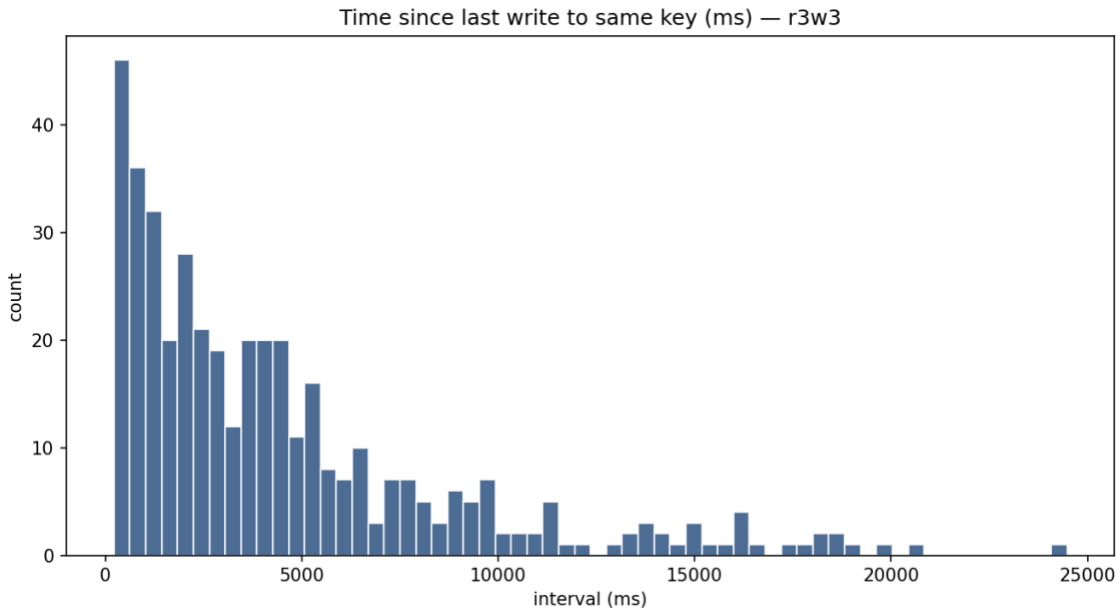


lf-r3w3-w0.5

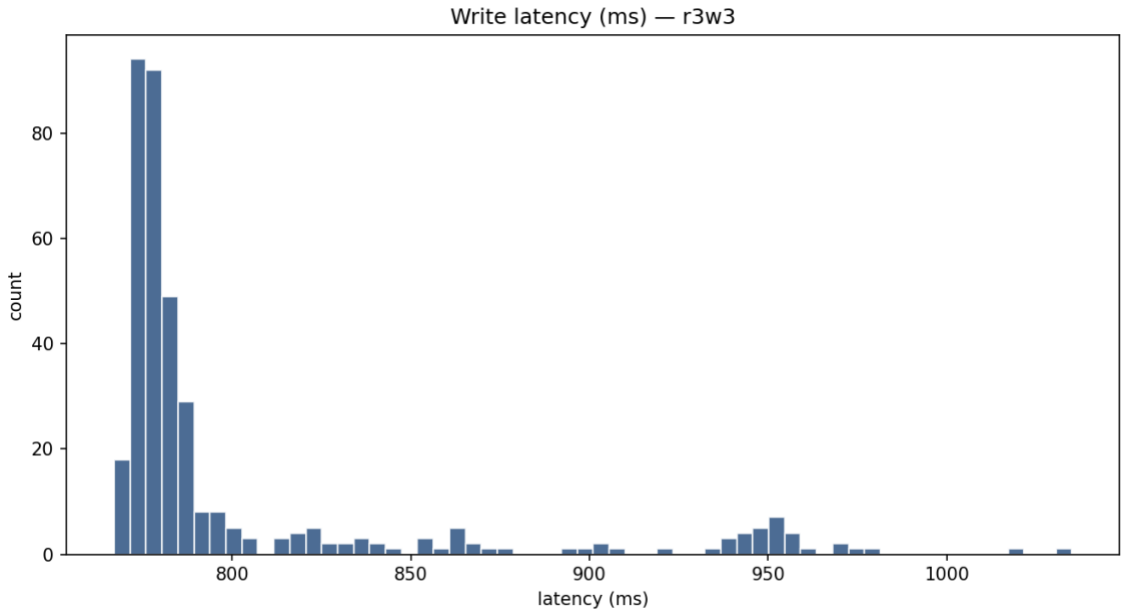
lf-r3w3-w0.5-read-latency.png



lf-r3w3-w0.5-rw-interval.png

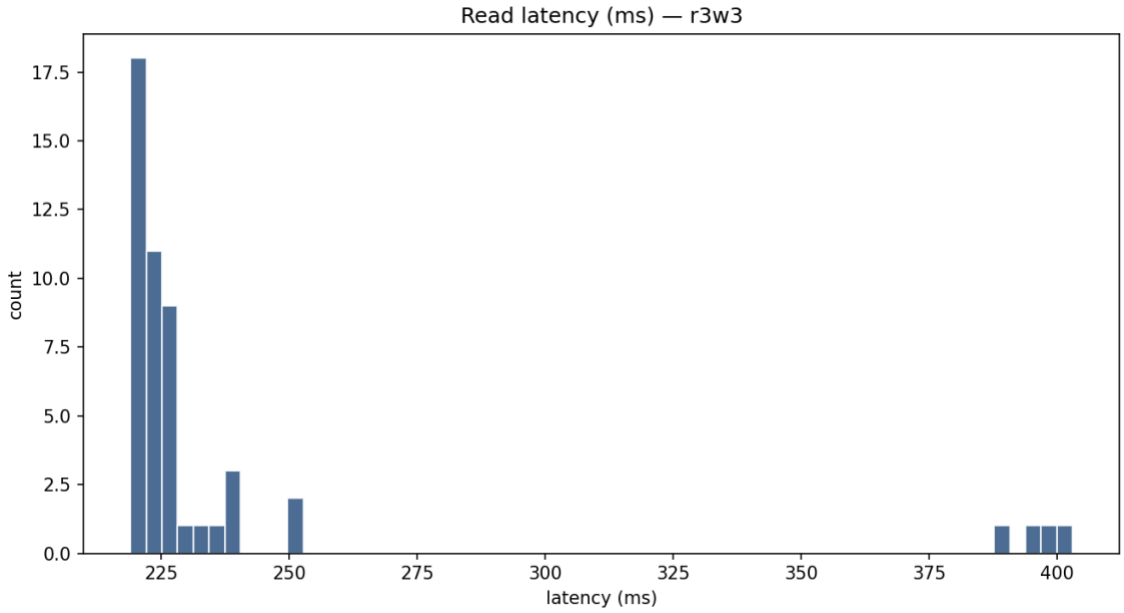


lf-r3w3-w0.5-write-latency.png

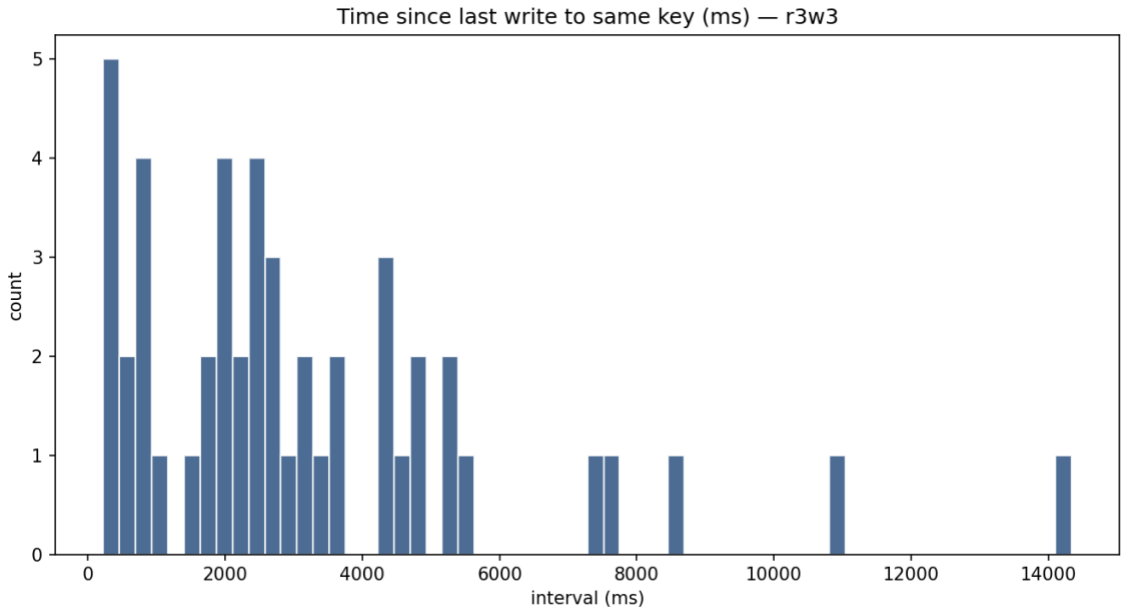


lf-r3w3-w0.9

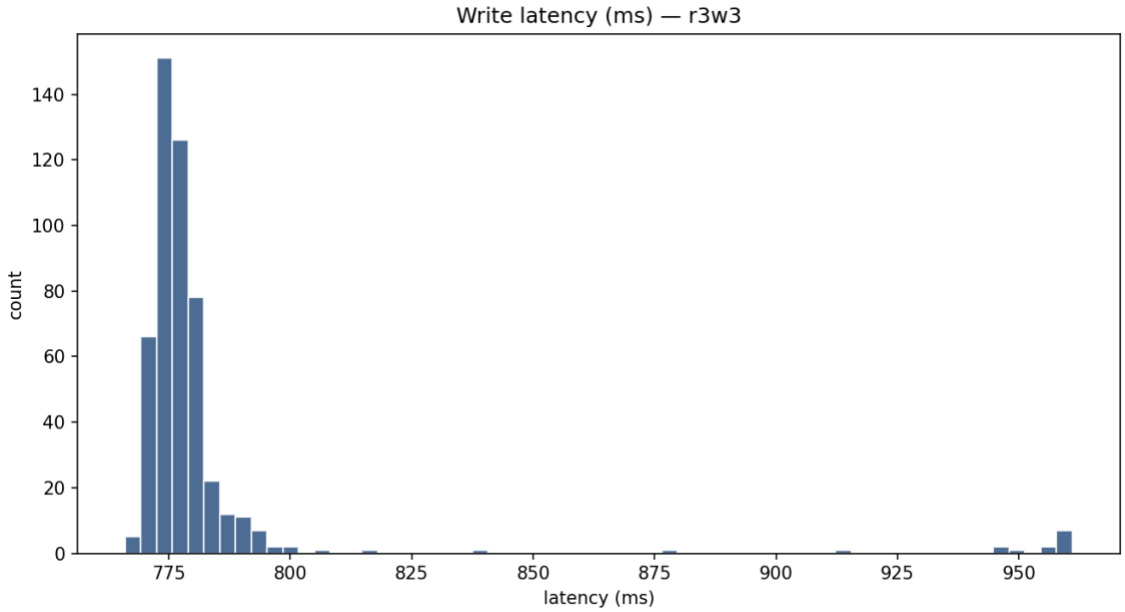
lf-r3w3-w0.9-read-latency.png



lf-r3w3-w0.9-rw-interval.png

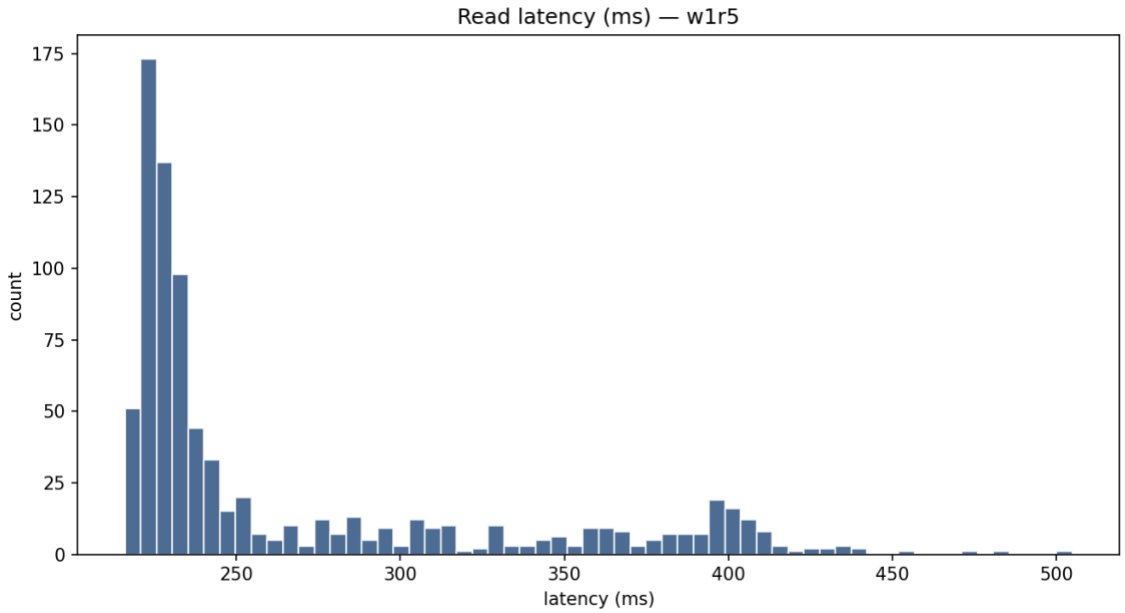


lf-r3w3-w0.9-write-latency.png

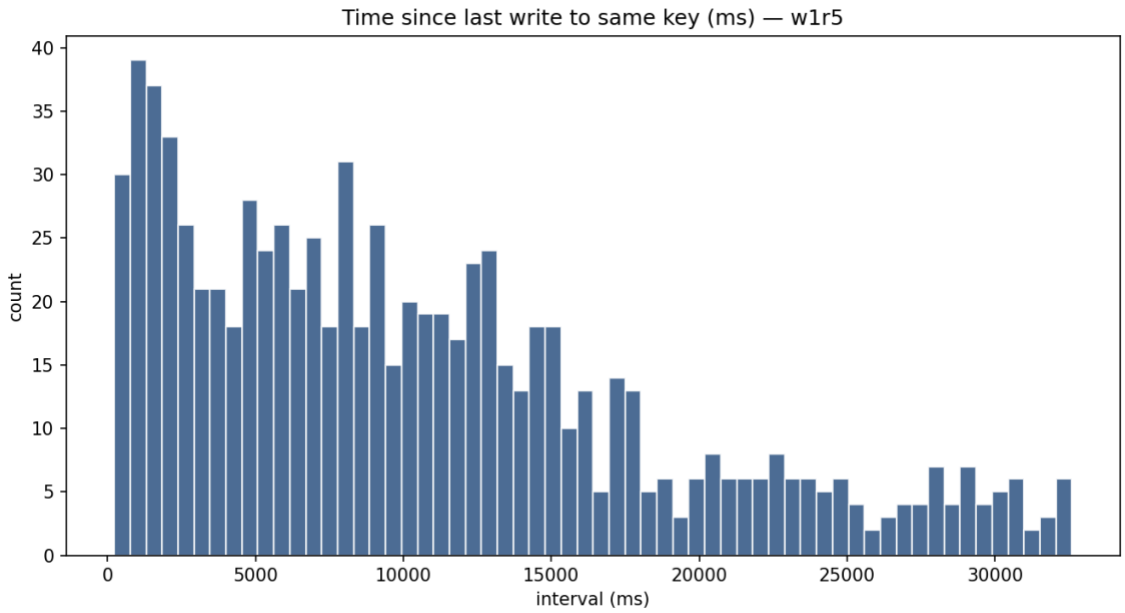


lf-w1r5-w0.01

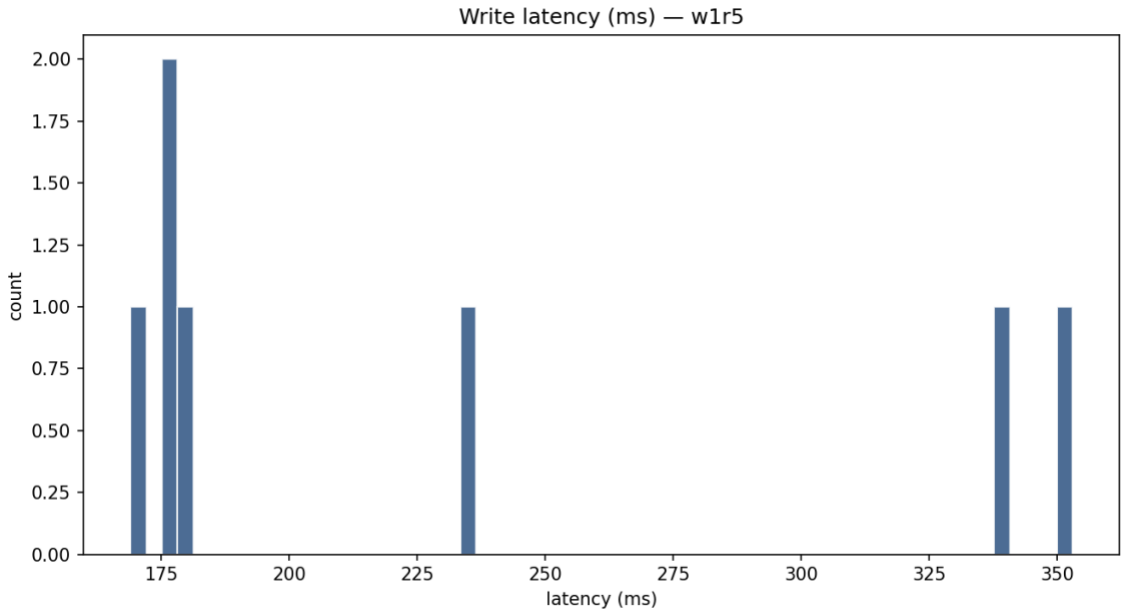
lf-w1r5-w0.01-read-latency.png



lf-w1r5-w0.01-rw-interval.png

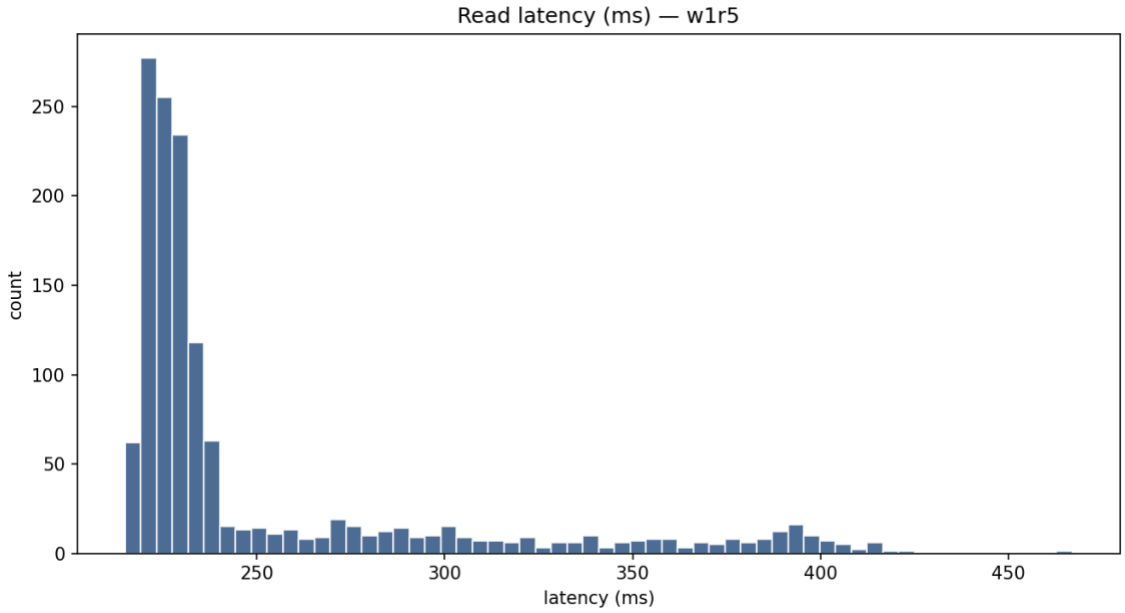


lf-w1r5-w0.01-write-latency.png

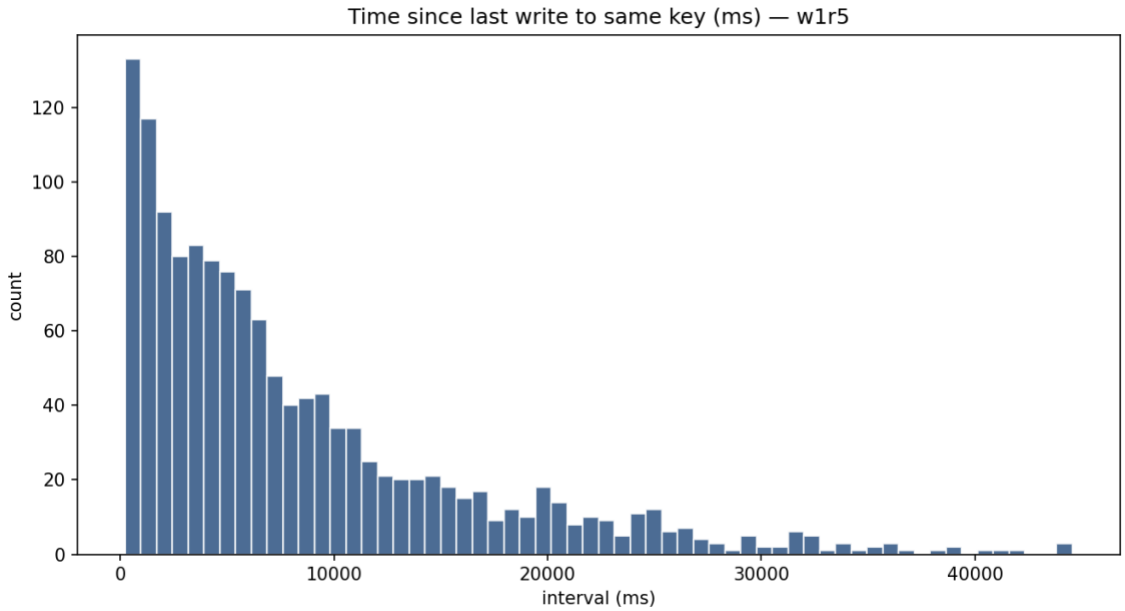


lf-w1r5-w0.1

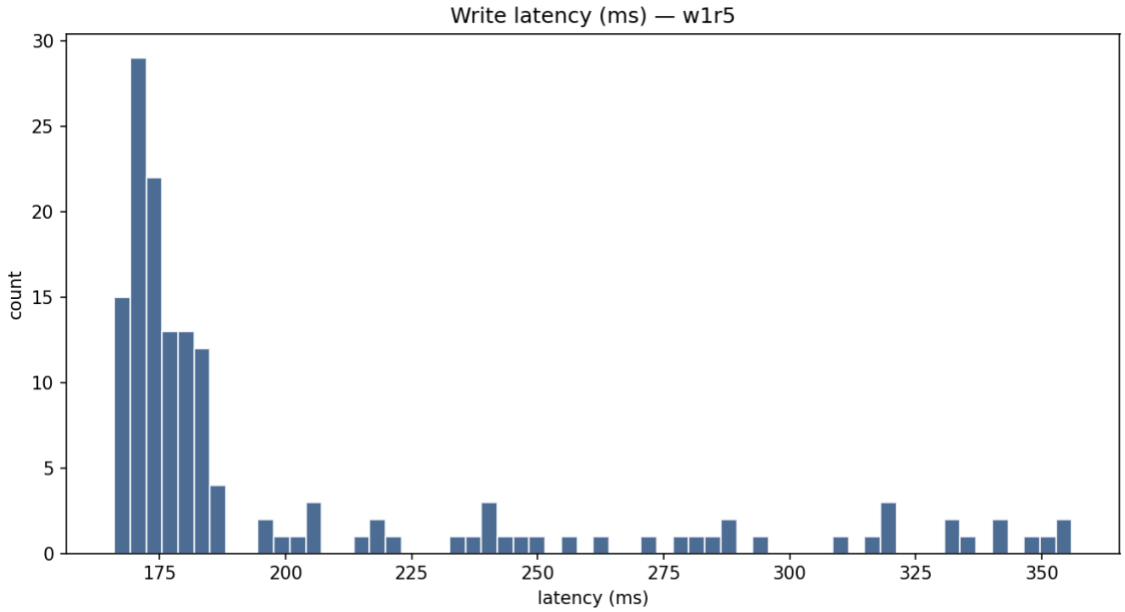
lf-w1r5-w0.1-read-latency.png



lf-w1r5-w0.1-rw-interval.png

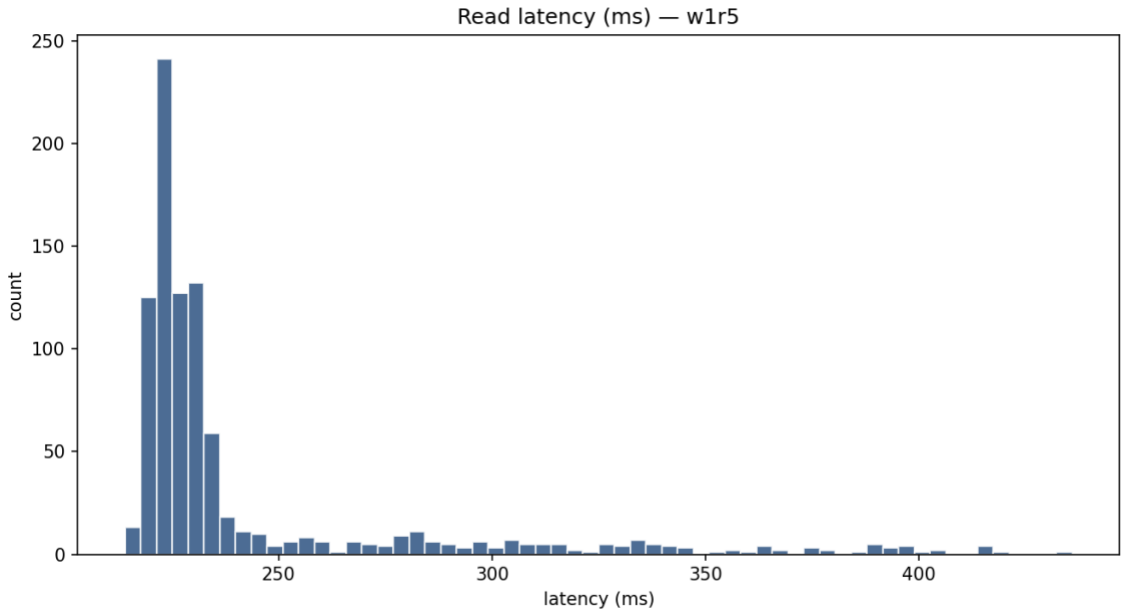


lf-w1r5-w0.1-write-latency.png

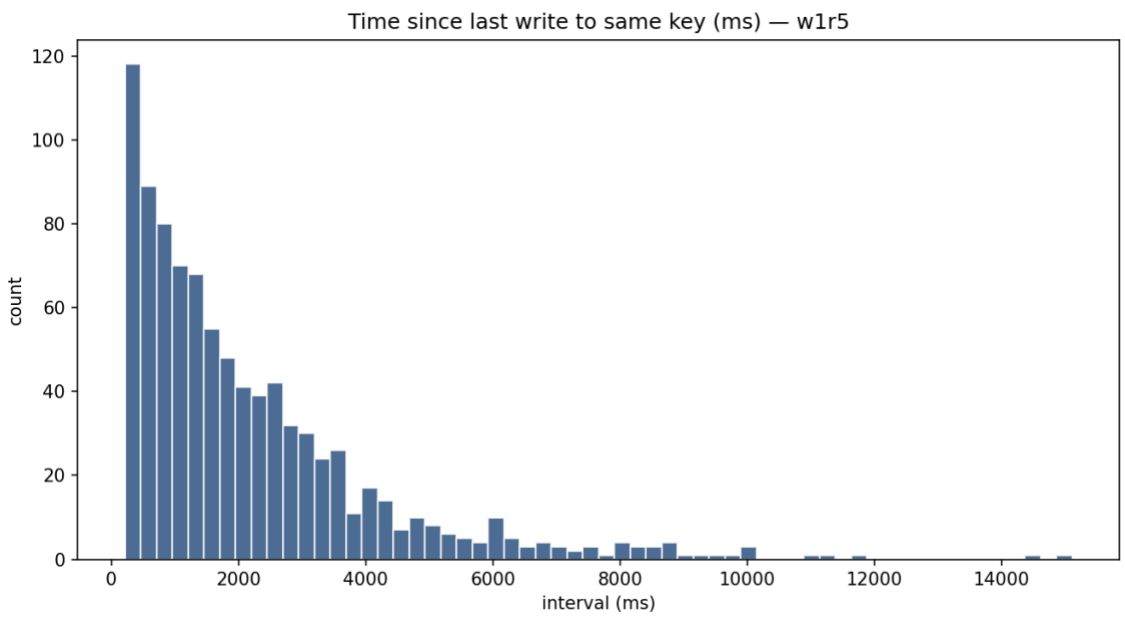


lf-w1r5-w0.5

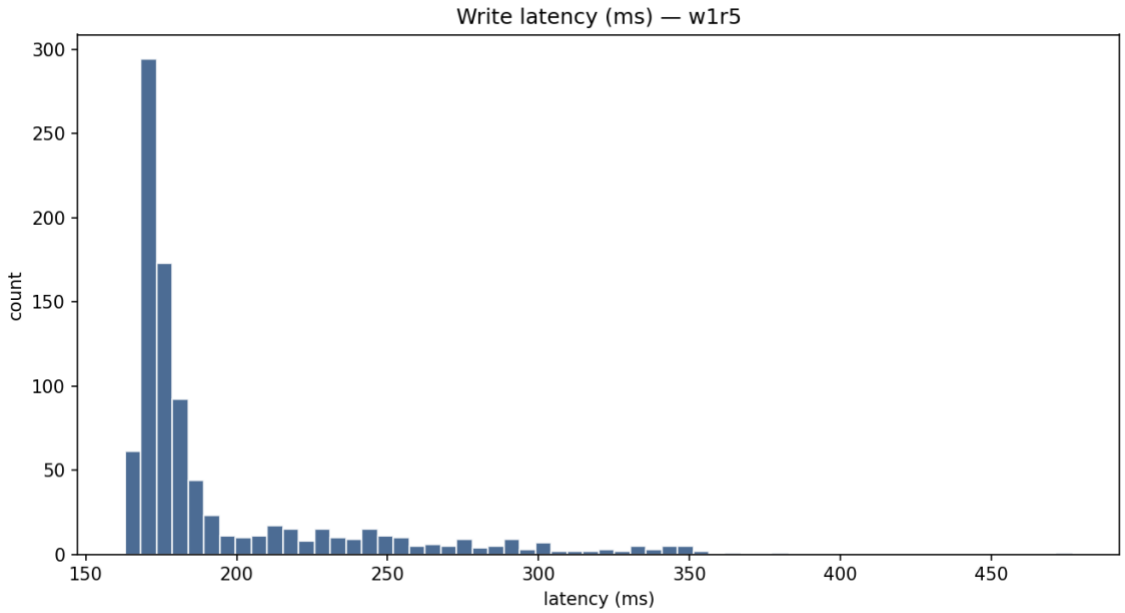
lf-w1r5-w0.5-read-latency.png



lf-w1r5-w0.5-rw-interval.png

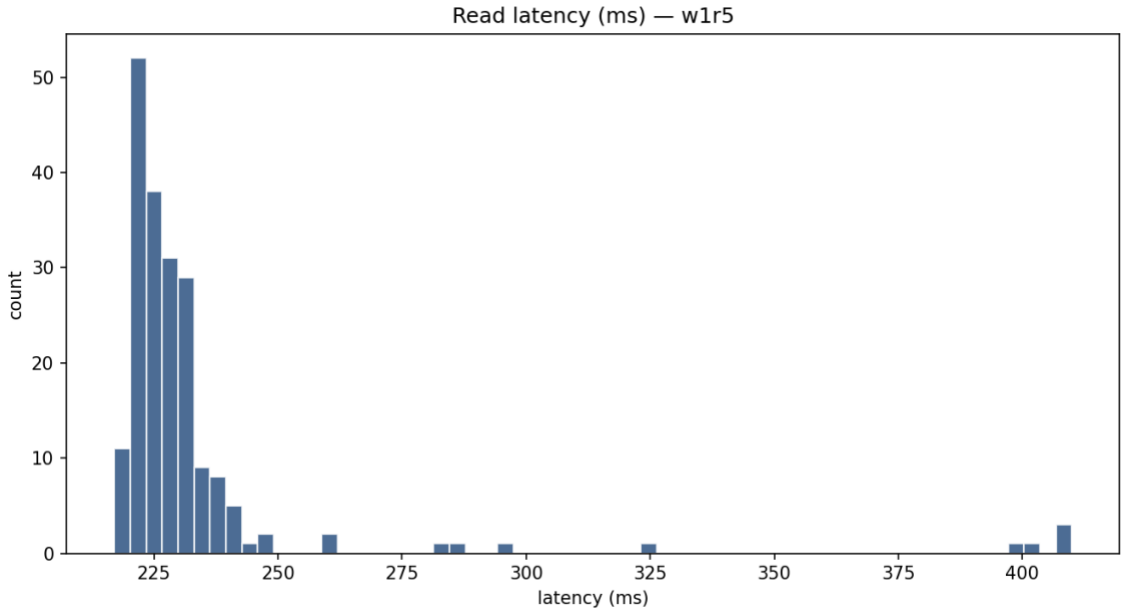


lf-w1r5-w0.5-write-latency.png

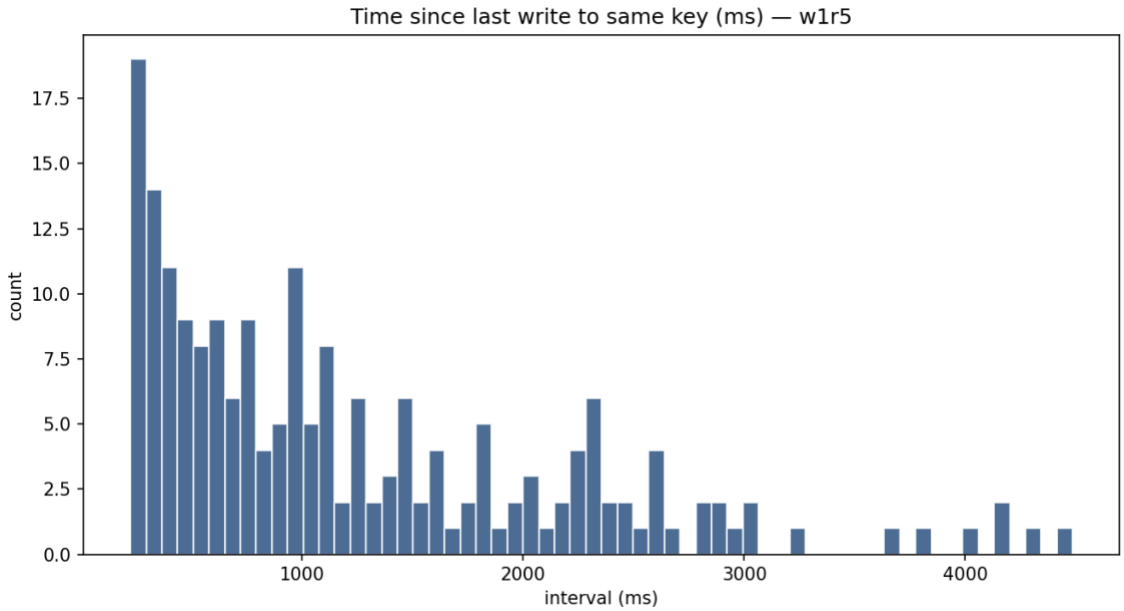


lf-w1r5-w0.9

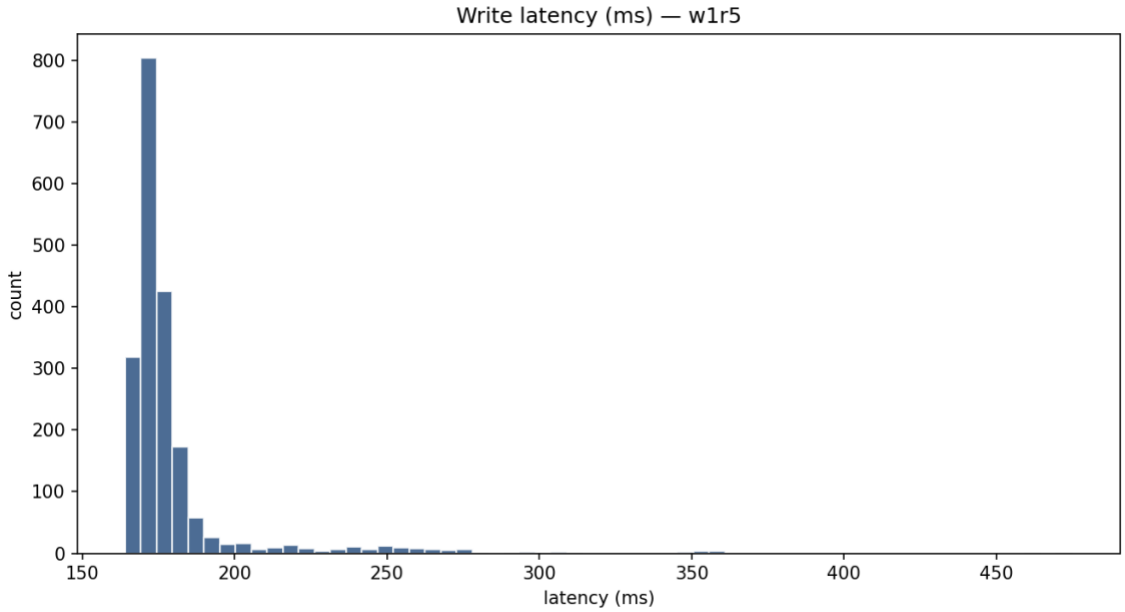
lf-w1r5-w0.9-read-latency.png



lf-w1r5-w0.9-rw-interval.png

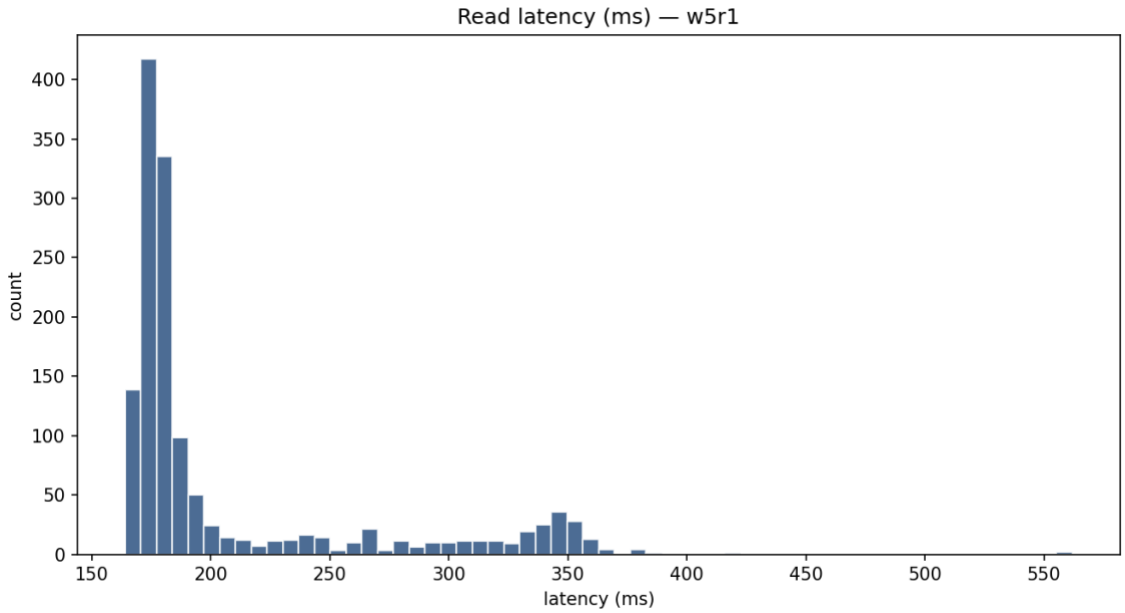


lf-w1r5-w0.9-write-latency.png

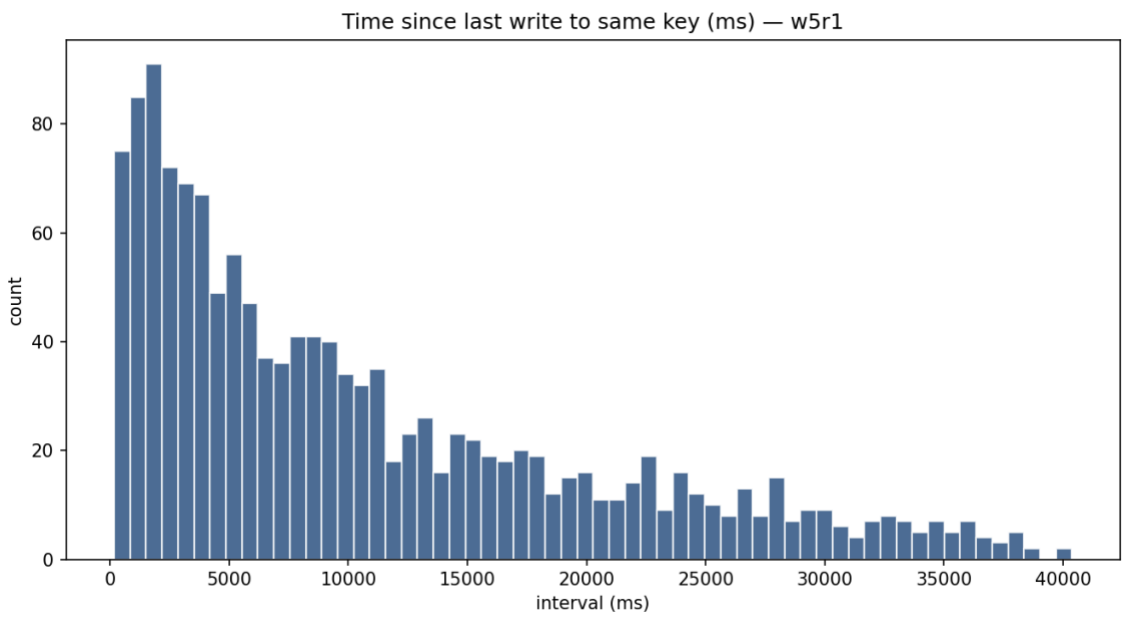


lf-w5r1-w0.01

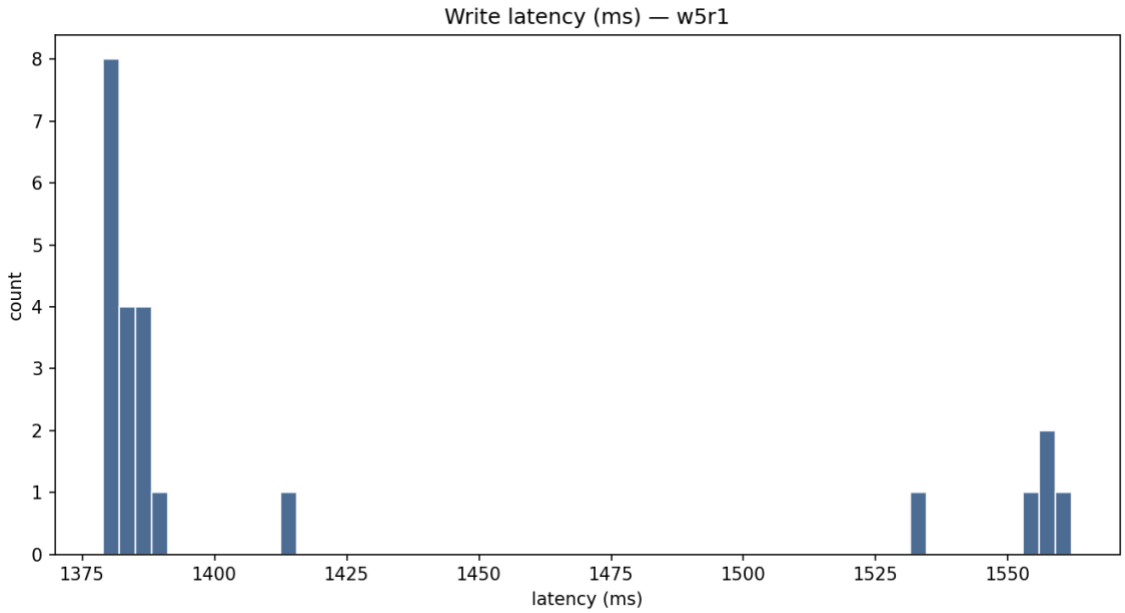
lf-w5r1-w0.01-read-latency.png



lf-w5r1-w0.01-rw-interval.png

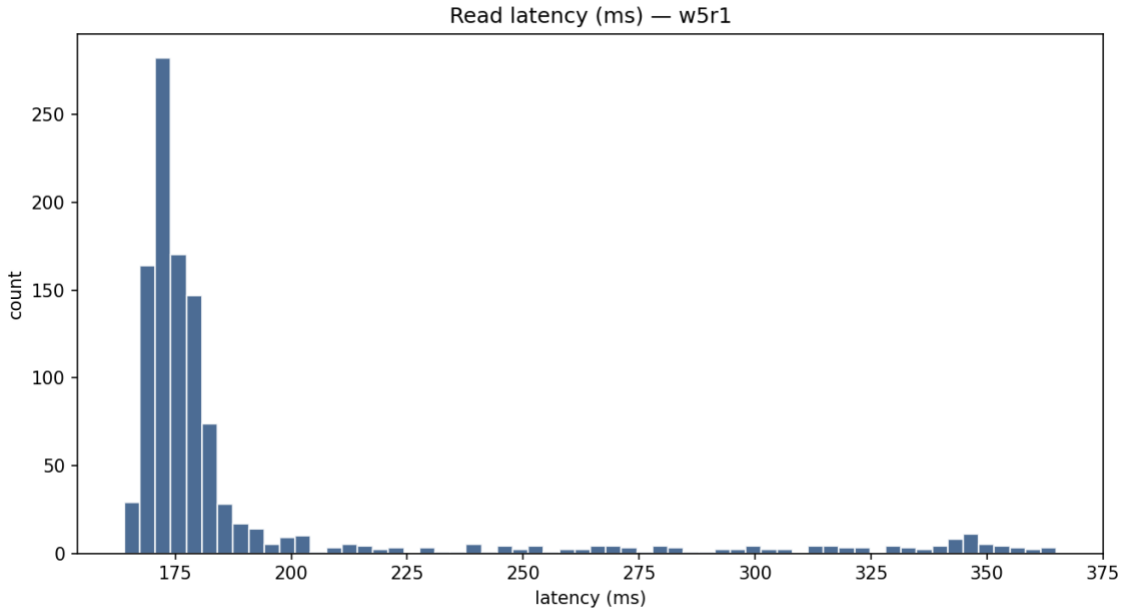


lf-w5r1-w0.01-write-latency.png

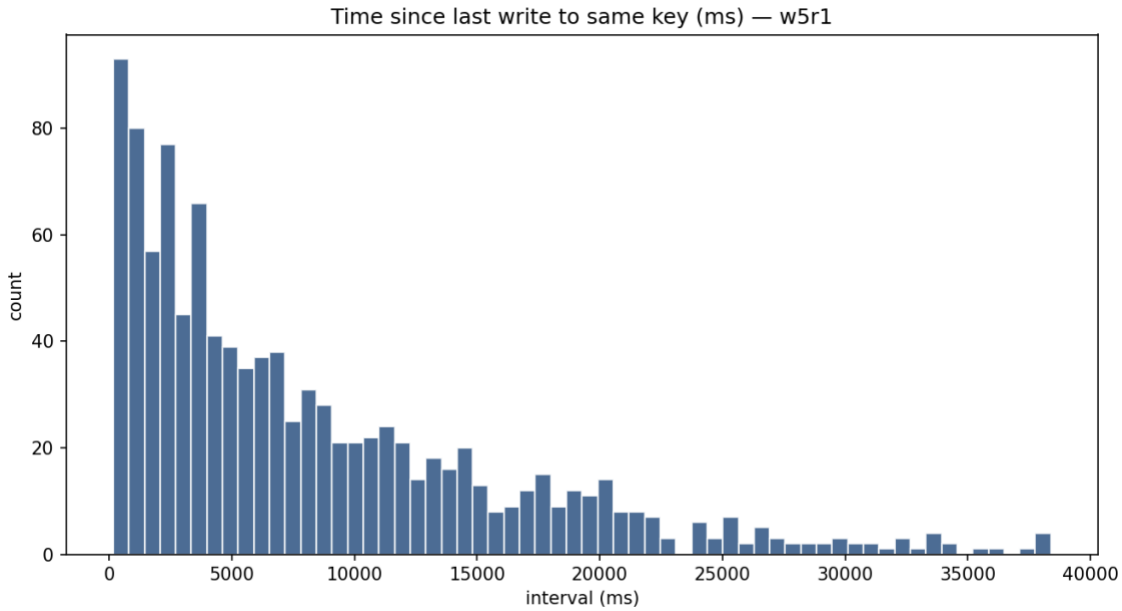


lf-w5r1-w0.1

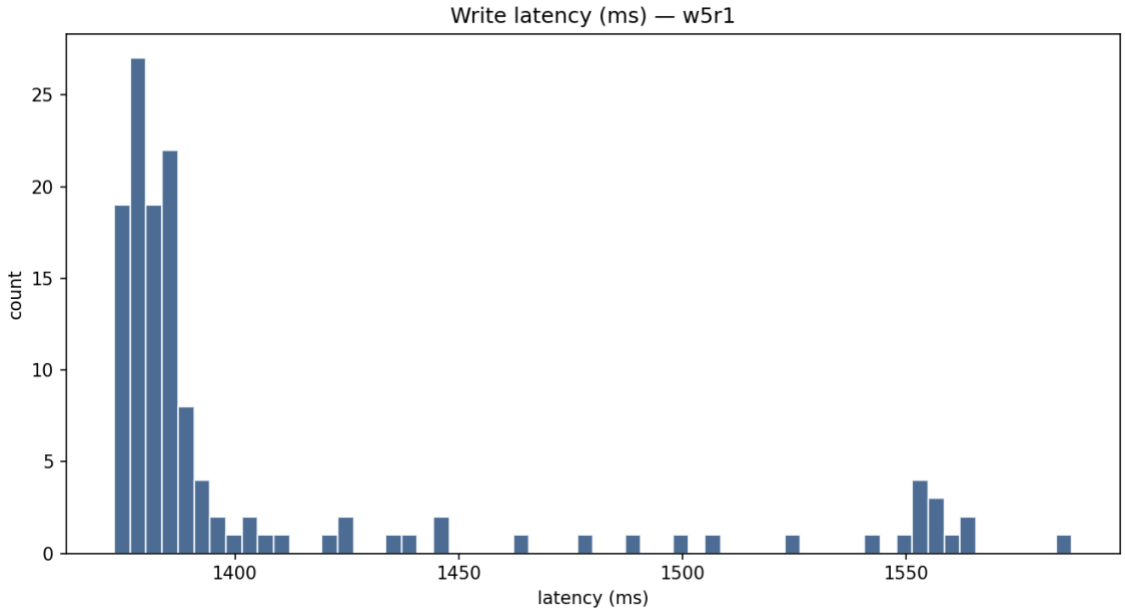
lf-w5r1-w0.1-read-latency.png



lf-w5r1-w0.1-rw-interval.png

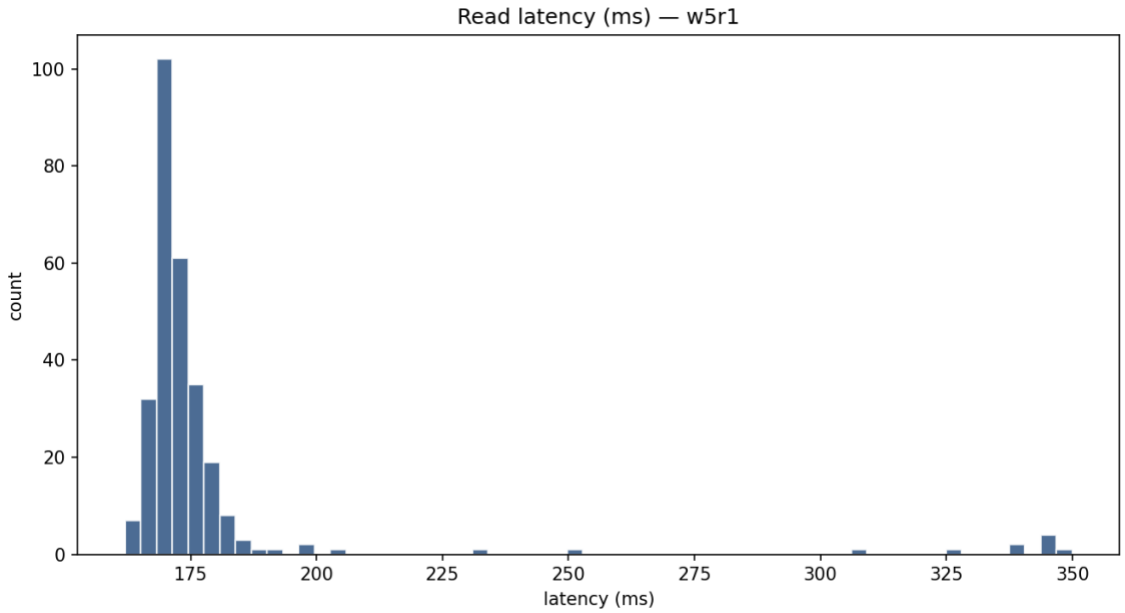


lf-w5r1-w0.1-write-latency.png

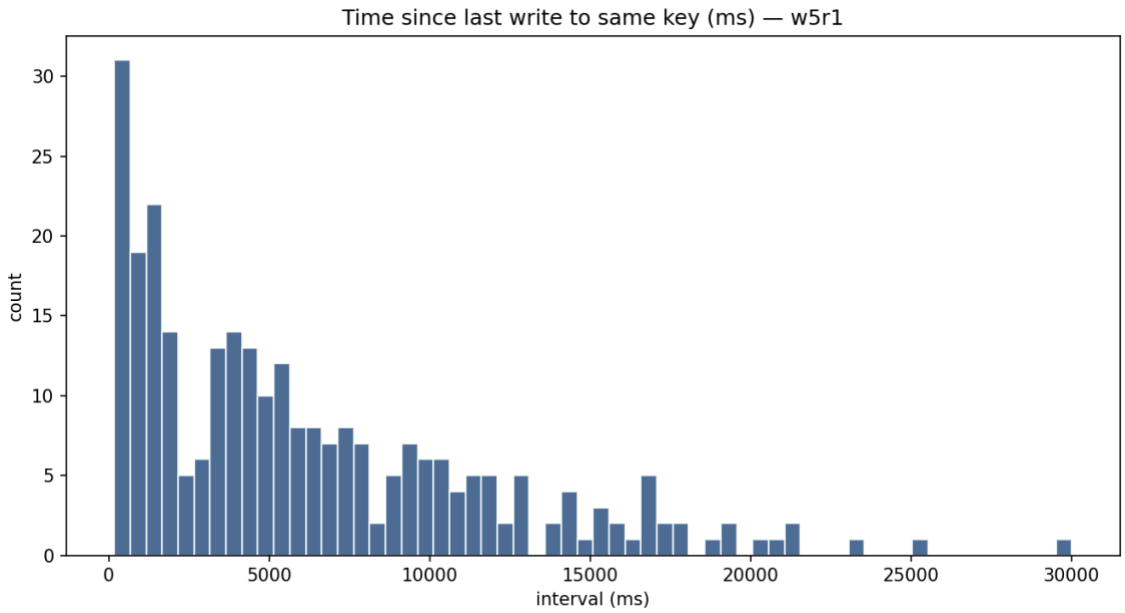


lf-w5r1-w0.5

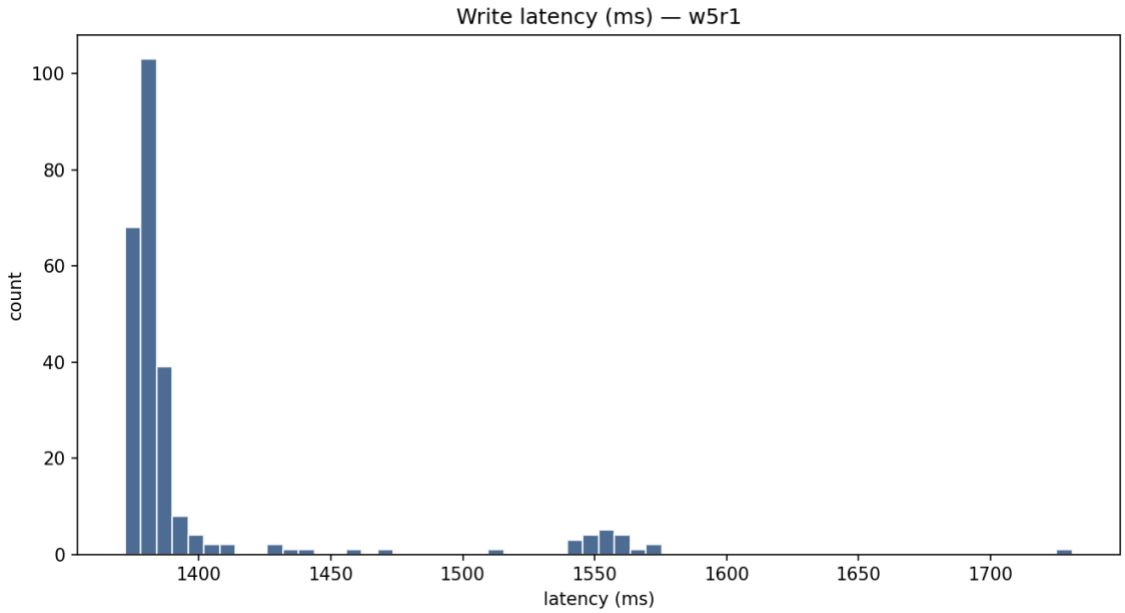
lf-w5r1-w0.5-read-latency.png



lf-w5r1-w0.5-rw-interval.png

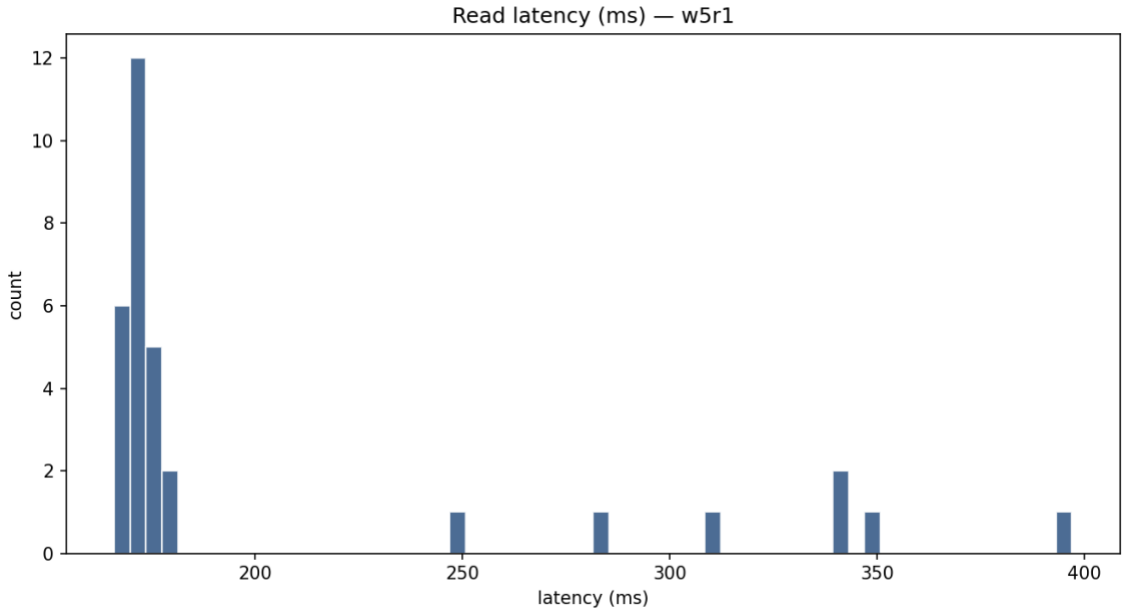


lf-w5r1-w0.5-write-latency.png

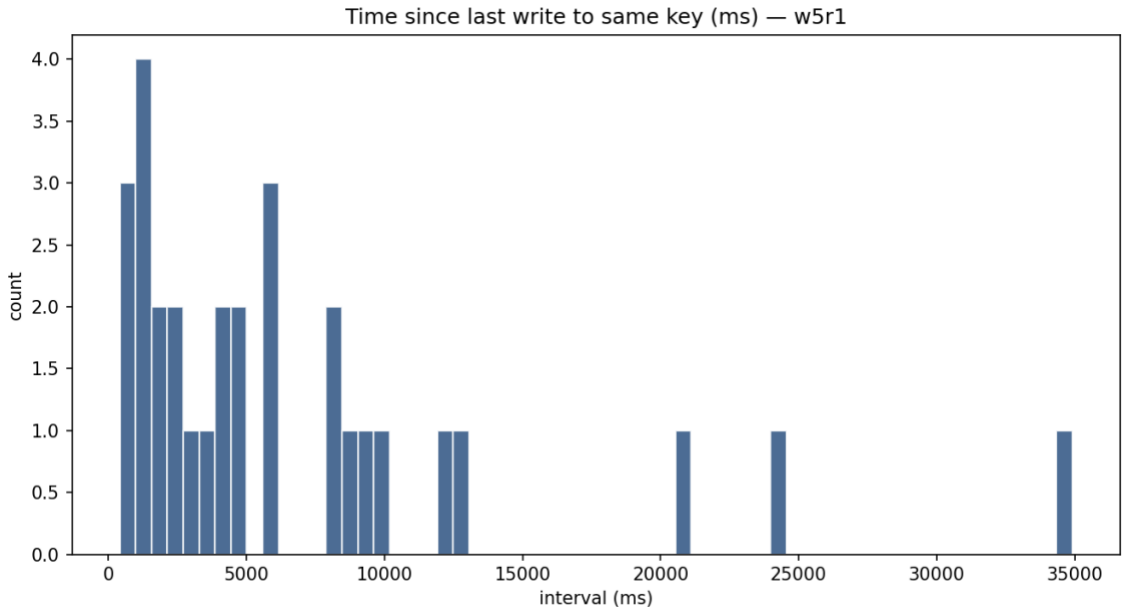


lf-w5r1-w0.9

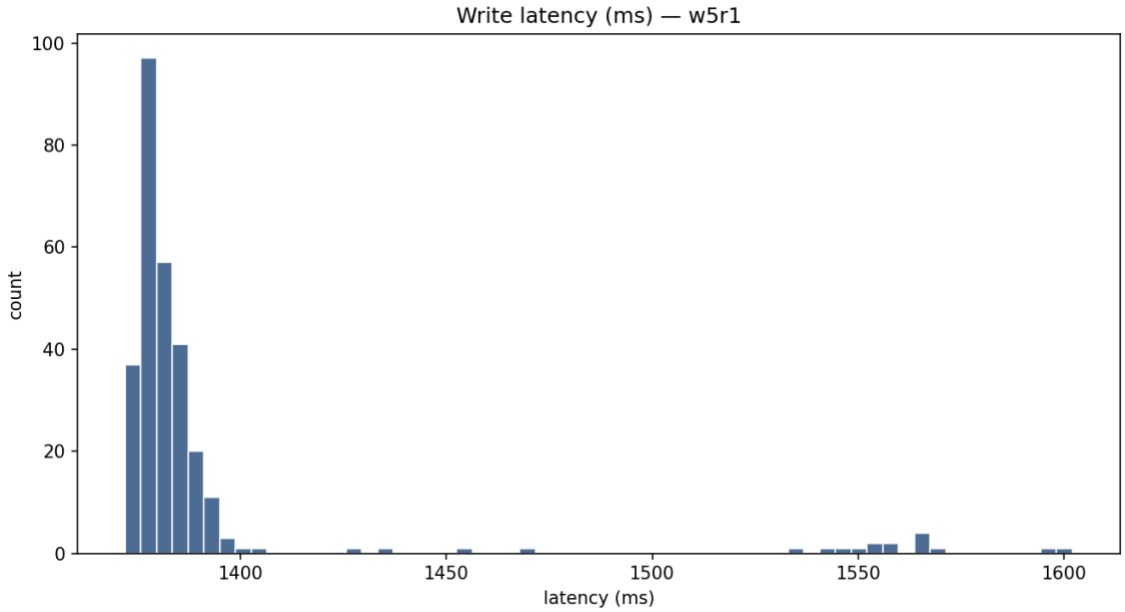
lf-w5r1-w0.9-read-latency.png



lf-w5r1-w0.9-rw-interval.png

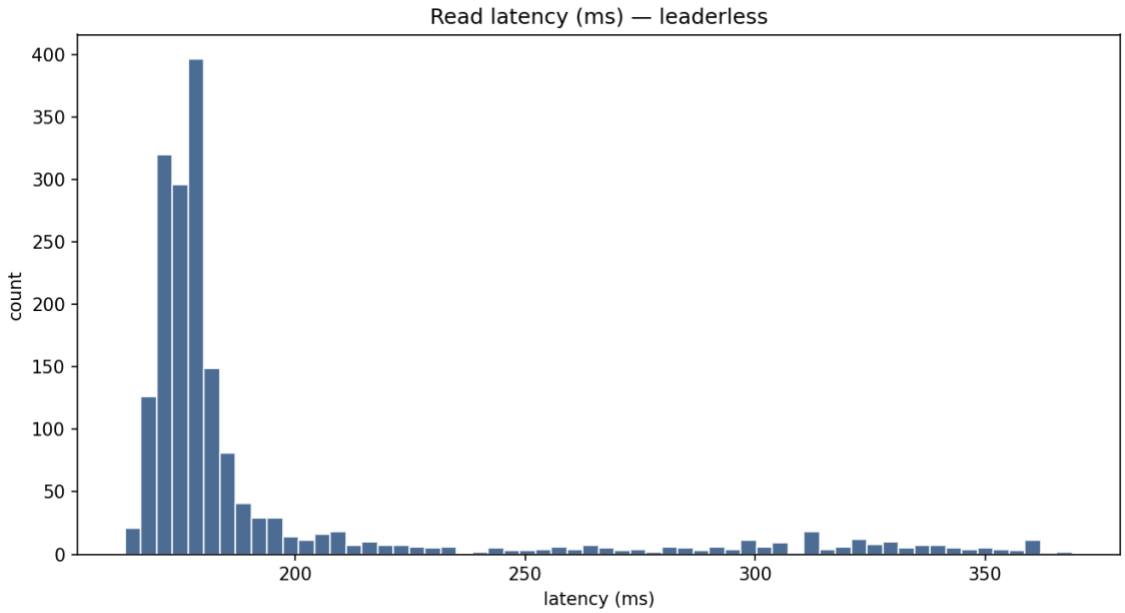


lf-w5r1-w0.9-write-latency.png

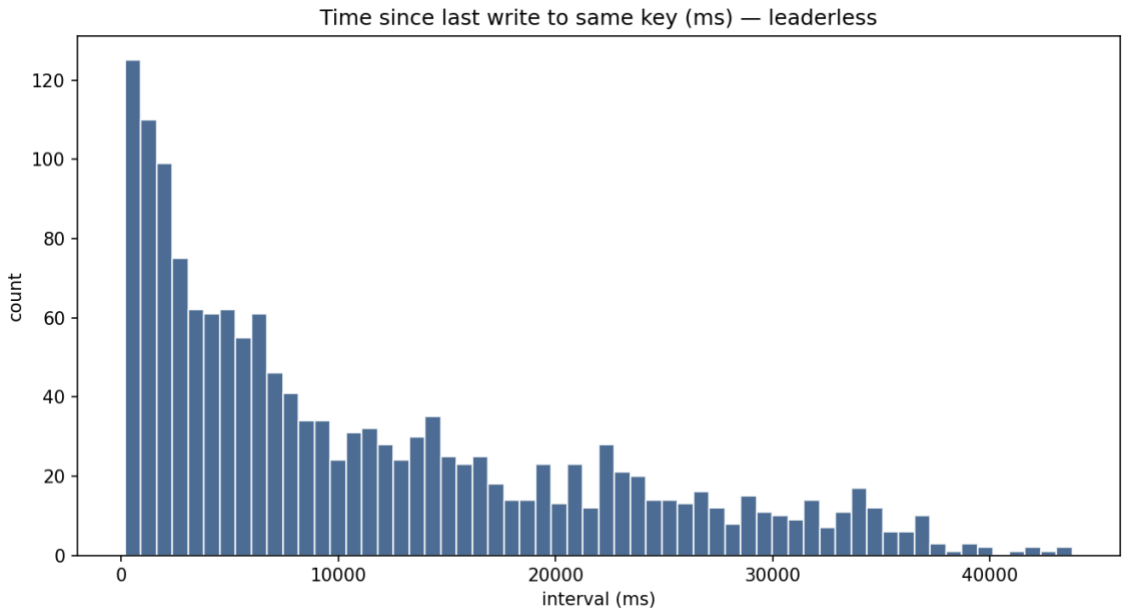


ll-w0.01

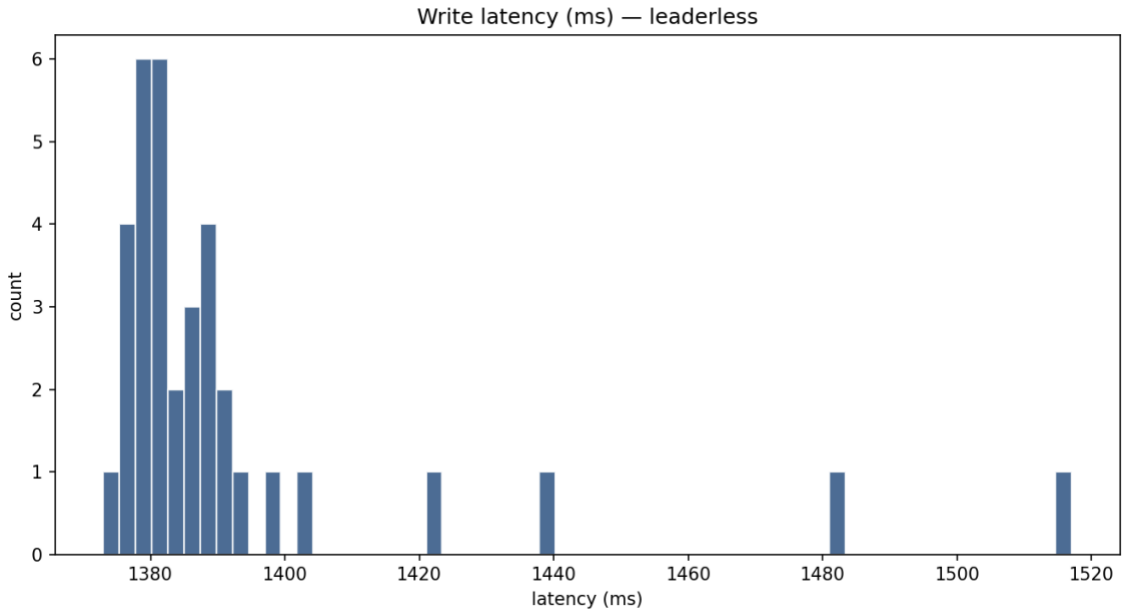
ll-w0.01-read-latency.png



ll-w0.01-rw-interval.png

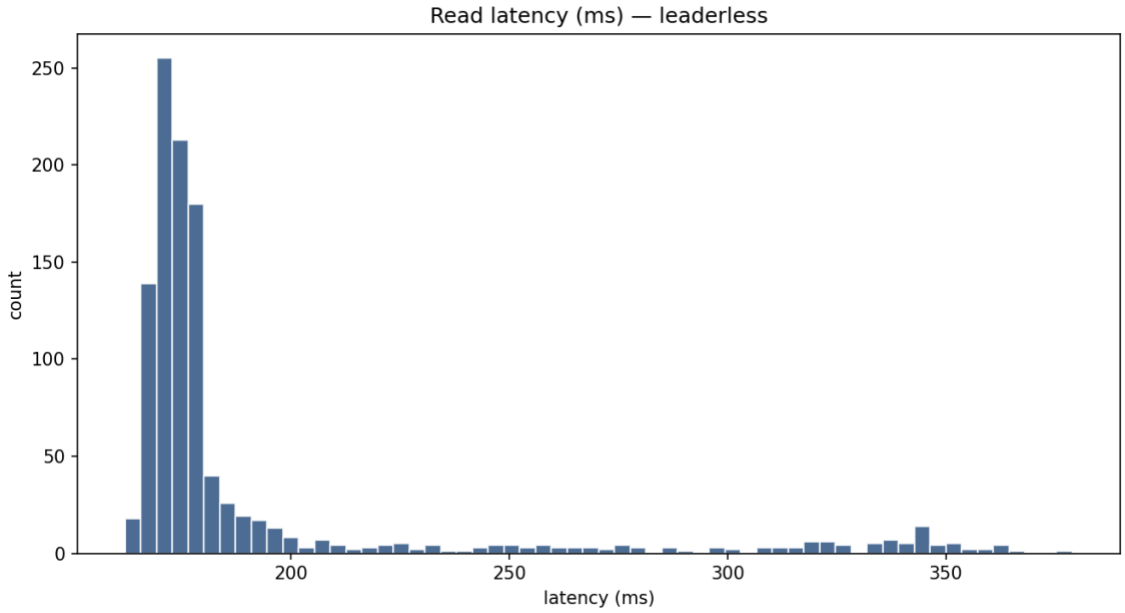


ll-w0.01-write-latency.png

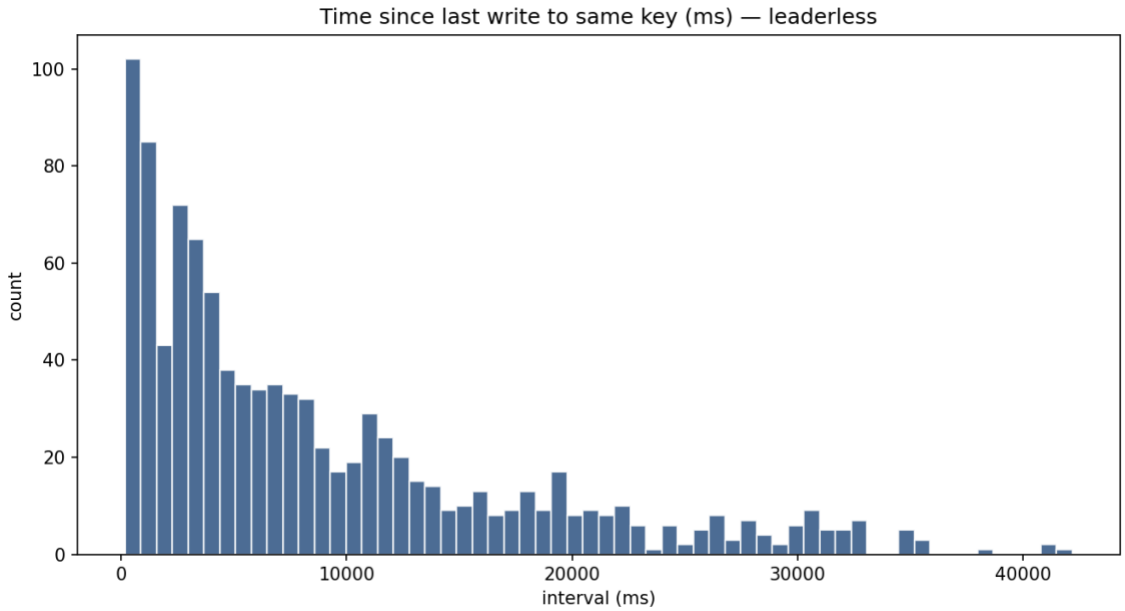


ll-w0.1

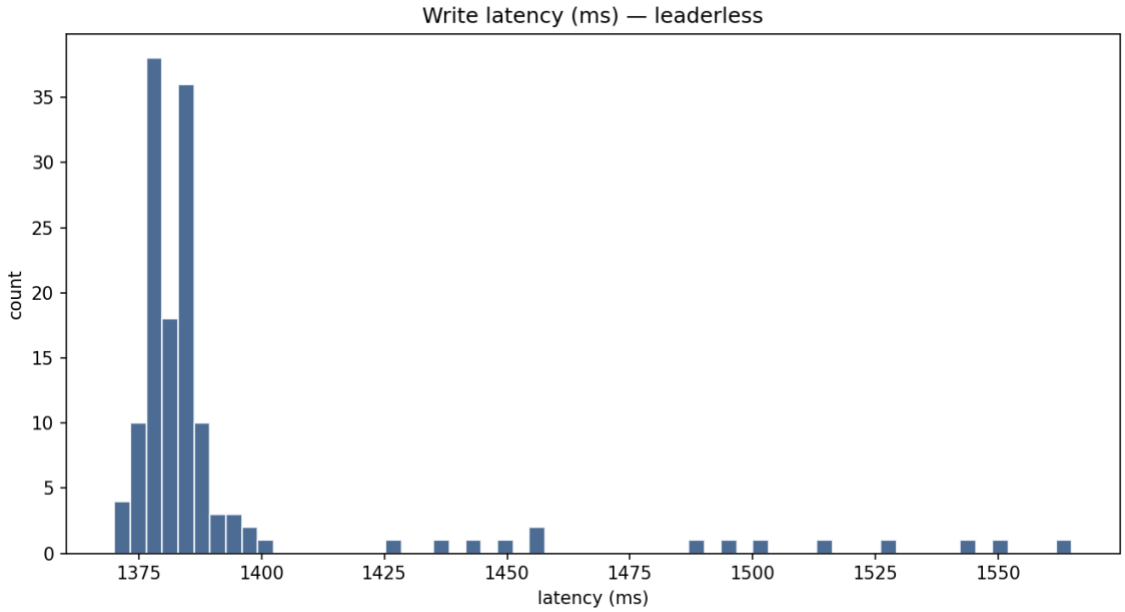
ll-w0.1-read-latency.png



ll-w0.1-rw-interval.png

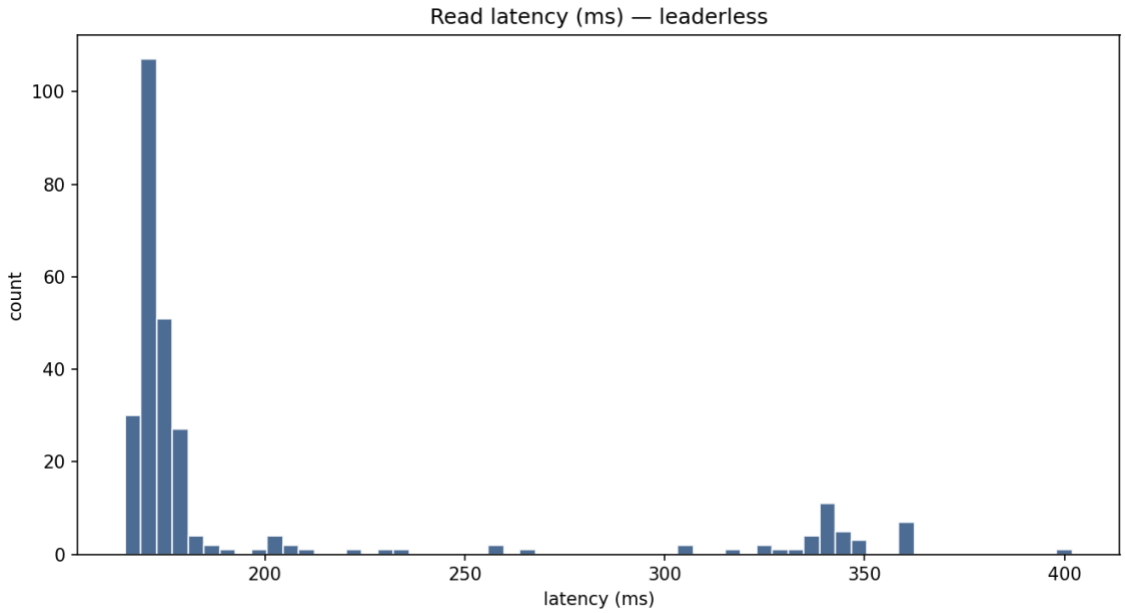


ll-w0.1-write-latency.png

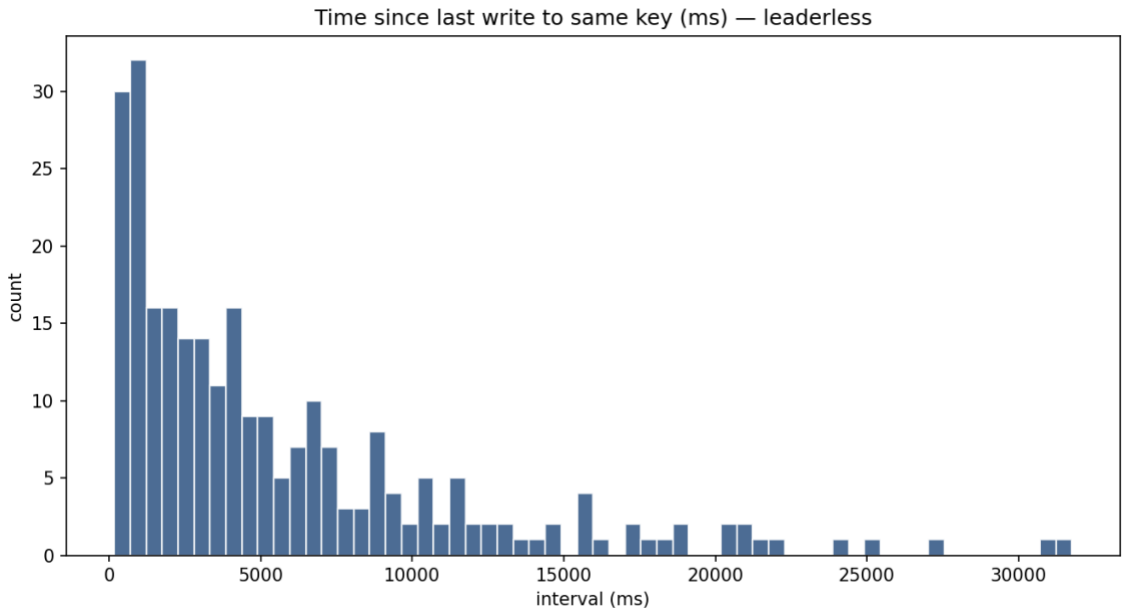


ll-w0.5

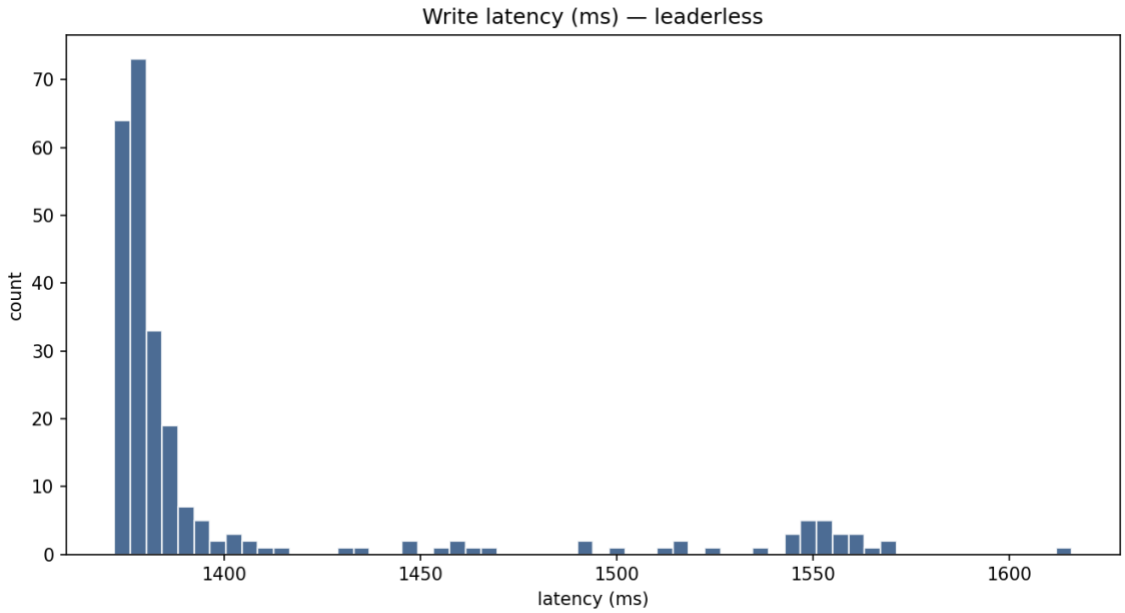
ll-w0.5-read-latency.png



ll-w0.5-rw-interval.png

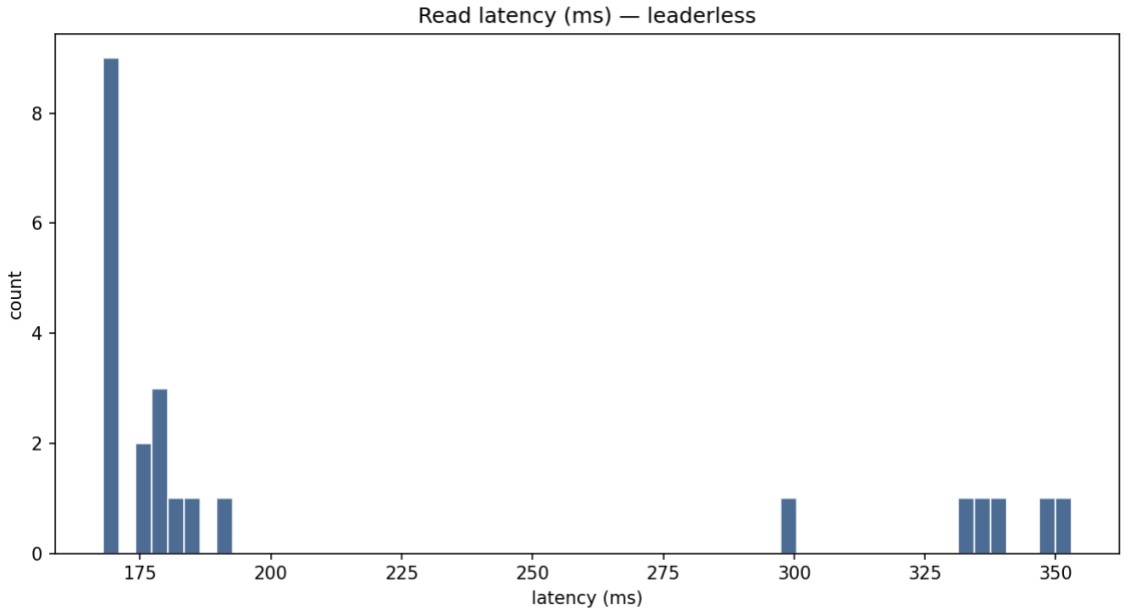


ll-w0.5-write-latency.png

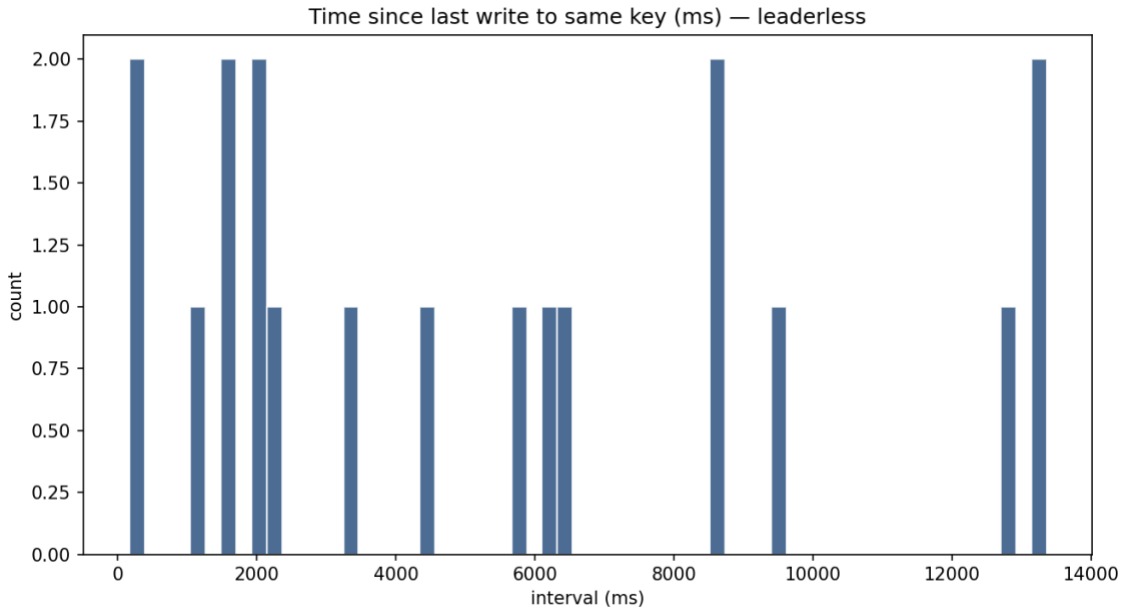
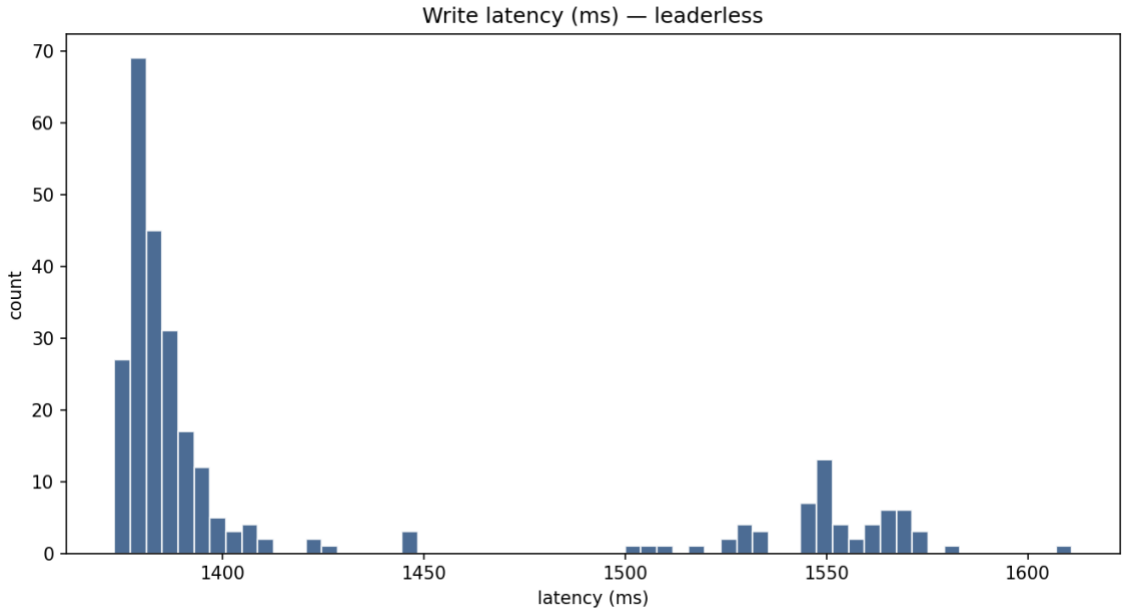


ll-w0.9

ll-w0.9-read-latency.png



ll-w0.9-rw-interval.png



ll-w0.9-write-latency.png